



UNIVERSIDADE FEDERAL DO CEARÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE TELEINFORMÁTICA (DETI)
CURSO DE GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

DANILO MOTA ALENCAR FILHO

**SISTEMA AUTOMATIZADO DE MONITORAMENTO DE TESTES DE
ENVELHECIMENTO ACELERADO PARA CIRCUITOS INTEGRADOS**

FORTALEZA

2026

DANILO MOTA ALENCAR FILHO

SISTEMA AUTOMATIZADO DE MONITORAMENTO DE TESTES DE
ENVELHECIMENTO ACELERADO PARA CIRCUITOS INTEGRADOS

Trabalho de Conclusão de Curso apresentado
ao Curso de Graduação em Engenharia de
Computação do Centro de Tecnologia da
Universidade Federal do Ceará, como requisito
parcial à obtenção do grau de bacharel em
Engenharia de Computação.

Orientador: Prof. Dr. Jarbas Aryel Nunes
da Silveira

FORTALEZA

2026

DANILO MOTA ALENCAR FILHO

SISTEMA AUTOMATIZADO DE MONITORAMENTO DE TESTES DE
ENVELHECIMENTO ACELERADO PARA CIRCUITOS INTEGRADOS

Trabalho de Conclusão de Curso apresentado
ao Curso de Graduação em Engenharia de
Computação do Centro de Tecnologia da
Universidade Federal do Ceará, como requisito
parcial à obtenção do grau de bacharel em
Engenharia de Computação.

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. Jarbas Aryel Nunes da Silveira (Orientador)
Universidade Federal do Ceará (UFC)

Prof. Me. Ricardo Jardel Nunes da Silveira
Universidade Federal do Ceará (UFC)

Prof. Dr. Alexandre Augusto da Penha Coelho
Universidade Federal do Ceará (UFC)

A Deus, por ser a minha razão de existir e perseverar, e à minha família, por serem o meu porto seguro. Irmã, sua tolerante companhia e a sua amizade me traz paz em nossa casa. Mãe, sua sabedoria e o seu carinho me inspiram a encarar todos os dias com amor em tudo que faço. Pai, os seus ensinamentos e o seu exemplo me motivam a seguir as veredas do justo.

AGRADECIMENTOS

Ao meu Senhor e Salvador, Jesus Cristo, pois eu sou testemunha da Tua Graça. Que a honra e a glória sejam sempre para engrandecer o Teu nome.

Ao Prof. Dr. Jarbas Aryel, por me permitir errar, aprender e me desenvolver sob a sua orientação, para além do que é apenas acadêmico e profissional.

Aos professores Jardel Silveira e Alexandre Coelho, que me deram meios de entrar no ambiente que mudou minha trajetória, como aluno e como engenheiro, sendo meus orientadores no meu primeiro projeto no Laboratório de Engenharia de Sistemas de Computação (LESC).

A todos os professores do Departamento de Engenharia Teleinformática (DETI), por exercerem esse nobre ofício que é a docência, e por serem exemplos para mim, não apenas como profissionais, mas também como integrantes da sociedade.

Aos meus professores dos tempos de escola, em especial o Tenente Belchior, do Colégio Militar de Brasília, e a Tenente Marina, do Colégio Militar de Fortaleza, por me inspirarem e me motivarem a sempre almejar mais, entregando o melhor que eu podia.

Aos meus amigos do laboratório, pelas referências que foram para mim durante essa trajetória e pelo suporte que me cederam em momentos de necessidade.

Aos meus companheiros de equipe, nas pesquisas e nos projetos, Alisson Jaime e Lucas Nogueira, que fazem o que for preciso para alcançar nossos objetivos coletivos, e por serem, além de grandes companheiros, grandes amigos meus.

À minha família, meus avós, meus tios, primos e aqueles que não compartilham laços de sangue, mas pelos quais eu nutro igualmente amor imenso, pelo carinho e pelo apoio que prestaram a mim. Vocês trouxeram as risadas, os conselhos e o acalento nos momentos que mais precisei.

À minha irmã, Ana Carolina, pela sua tolerância comigo durante esse período, e por ter feito a nossa casa ser um lar pra mim. Essa conquista é nossa, e eu não poderia tê-la sem o teu apoio.

Por fim, agradeço ao meu pai e à minha mãe, Danilo e Katerine, por terem me rodeado de amor infindável e me criado com valores, de forma que o mundo pudesse me dobrar, mas que os meus princípios permanecessem firmes.

“So we beat on, boats against the current, borne
back ceaselessly into the past.”

(The Great Gatsby, F. Scott Fitzgerald)

RESUMO

A confiabilidade de circuitos integrados, especialmente *Field-Programmable Gate Arrays* (FPGAs) fabricadas em tecnologias nanométricas, é desafiada por mecanismos de envelhecimento como *Bias Temperature Instability* (BTI) e *Hot Carrier Injection* (HCI). Este trabalho apresenta o desenvolvimento de uma plataforma automatizada e de baixo custo para testes de vida acelerada (*burn-in*), visando a validação de sensores de envelhecimento *on-chip* no contexto do *Silicon Lifecycle Management* (SLM). O sistema integra uma câmara térmica controlada por um algoritmo *proportional-integral-derivative* (PID) digital, com sintonia inicial pelo método *Simple Internal Model Control* (SIMC) e refinamento empírico, executado em um microcontrolador Arduino Nano 33 IoT com conversor analógico-digital (*analog-to-digital converter* – ADC) de 12 bits. Uma aplicação supervisória em Python, baseada em arquitetura *multi-threading*, orquestra o estresse térmico e elétrico simultaneamente à coleta de dados do sensor. Os resultados experimentais evidenciaram melhoria significativa da estabilidade térmica com controle PID em relação ao controle On-Off, reduzindo o desvio padrão em regime permanente para $\sigma = 0,40^{\circ}\text{C}$ e limitando o erro máximo a $|e|_{\max} = 1,8^{\circ}\text{C}$, além de validar a arquitetura de software e a capacidade de rastreamento do sensor.

Palavras-chave: Confiabilidade. FPGA. *Burn-in*. Controle PID. Automação.

ABSTRACT

The reliability of integrated circuits, especially *field-programmable gate arrays* (FPGAs) manufactured in nanometric technologies, is challenged by aging mechanisms such as *Bias Temperature Instability* (BTI) and *Hot Carrier Injection* (HCI). This work presents the development of a low-cost, automated platform for accelerated life testing (*burn-in*), aimed at validating *on-chip* aging sensors within the context of *Silicon Lifecycle Management* (SLM). The system integrates a thermal chamber controlled by a digital *proportional-integral-derivative* (PID) algorithm, with initial tuning based on the *Simple Internal Model Control* (SIMC) method and subsequent empirical refinement, executed on an Arduino Nano 33 IoT microcontroller with a 12-bit *analog-to-digital converter* (ADC). A Python-based supervisory application, built on a *multi-threading* architecture, orchestrates simultaneous thermal and electrical stress while collecting sensor data. Experimental results showed a marked improvement in thermal stability when using PID control compared to On-Off control, achieving a steady-state standard deviation of $\sigma = 0.40^{\circ}\text{C}$ and limiting the maximum error to $|e|_{\max} = 1.8^{\circ}\text{C}$, while validating both the software architecture and the sensor's tracking capability.

Keywords: Reliability. FPGA. *Burn-in*. PID control. Automation.

LISTA DE FIGURAS

Figura 1 – Representação da arquitetura de sensores de caminho crítico baseados em flip-flops.	28
Figura 2 – Diagrama de temporização mostrando o comportamento esperado do <i>slack</i> devido em um circuito novo.	29
Figura 3 – Diagrama de temporização mostrando a erosão do <i>slack</i> devido ao envelhecimento do circuito.	29
Figura 4 – Diagrama de blocos do sistema automatizado de testes de envelhecimento. .	35
Figura 5 – Resposta ao degrau do forno para identificação dos parâmetros <i>First-Order Plus Dead Time</i> (FOPDT). A linha verde tracejada indica o tempo morto, a linha vermelha indica 63.2% da subida, e a linha roxa indica o tempo em que essa temperatura foi atingida.	39
Figura 6 – Tela da IHM desenvolvida em meio a execução de um teste. À esquerda, o gráfico de monitoramento das características térmicas do sistema e, à direita, o gráfico de monitoramento das características elétricas do mesmo.	45
Figura 7 – Análise comparativa entre controlador <i>Skogestad Internal Model Control</i> (SIMC) puro (vermelho) e controlador após <i>fine-tuning</i> (azul) para setpoint de 100°C. (a) Evolução da temperatura; (b) Regime permanente em detalhe; (c) Esforço de controle; (d) Distribuição de temperatura em regime permanente; (e) Erro de controle; (f) Boxplot dos dois conjuntos.	47
Figura 8 – Análise comparativa entre controle <i>Proportional-Integral-Derivative</i> (PID) (azul) e controle Bang-Bang/On-Off (vermelho) para setpoint de 100°C. (a) Evolução da temperatura; (b) Regime permanente em detalhe; (c) Esforço de controle; (d) Distribuição de temperatura; (e) Erro de controle; (f) Boxplot dos dois conjuntos.	48
Figura 9 – Análise comparativa entre sistema SBCCI original (vermelho) e sistema PID proposto (azul) durante teste de 24 horas. (a) Temperatura do <i>Device Under Test</i> (DUT) ao longo do tempo; (b) Distribuição de temperatura; (c) Tensão do núcleo; (d) Evolução do <i>slack</i> (indicador de envelhecimento); (e-f) Boxplot da temperatura e do <i>slack</i> dos testes	50

LISTA DE TABELAS

Tabela 1 – Comparativo entre estratégias de controle térmico	32
Tabela 2 – Parâmetros FOPDT identificados experimentalmente	39
Tabela 3 – Comparativo entre ganhos SIMC teóricos e após <i>fine-tuning</i>	40
Tabela 4 – Comparativo de ganhos SIMC teórico vs. <i>fine-tuned</i>	46
Tabela 5 – Comparativo estatístico: SIMC teórico vs. <i>fine-tuned</i>	46
Tabela 6 – Comparativo estatístico de estabilidade térmica: PID vs. On-Off	49
Tabela 7 – Comparativo estatístico: Sistema SBCCI vs. Sistema PID Proposto (24h) . .	51
Tabela 8 – Exemplo de amostra de dados consolidados pelo sistema	51
Tabela 9 – Avaliação do cumprimento dos objetivos específicos	54

LISTA DE ABREVIATURAS E SIGLAS

ADC	<i>Analog-to-Digital Converter</i>
ALT	<i>Accelerated Life Testing</i>
ATE	<i>Automated Test Equipment</i>
BGA	<i>Ball Grid Array</i>
BTI	<i>Bias Temperature Instability</i>
CMOS	<i>Complementary Metal-Oxide-Semiconductor</i>
COTS	<i>Commercial Off-The-Shelf</i>
CSV	<i>Comma-Separated Values</i>
DUT	<i>Device Under Test</i>
FOPDT	<i>First-Order Plus Dead Time</i>
FPGA	<i>Field-Programmable Gate Array</i>
HCI	<i>Hot Carrier Injection</i>
HMI	<i>Human-Machine Interface</i>
HTOL	<i>High Temperature Operating Life</i>
IHM	<i>Interface Homem-Máquina</i>
JEDEC	<i>Joint Electron Device Engineering Council</i>
JTAG	<i>Joint Test Action Group</i>
MCU	<i>Microcontroller Unit</i>
MMCM	<i>Mixed-Mode Clock Manager</i>
NBTI	<i>Negative Bias Temperature Instability</i>
NTC	<i>Negative Temperature Coefficient</i>
PBTI	<i>Positive Bias Temperature Instability</i>
PID	<i>Proportional-Integral-Derivative</i>
PSU	<i>Power Supply Unit</i>
PVT	<i>Process, Voltage, and Temperature</i>
PWM	<i>Pulse Width Modulation</i>
RUL	<i>Remaining Useful Life</i>
SCPI	<i>Standard Commands for Programmable Instruments</i>
SIMC	<i>Skogestad Internal Model Control</i>
SLM	<i>Silicon Lifecycle Management</i>
SSR	<i>Solid State Relay</i>

TID	<i>Total Ionizing Dose</i>
UART	<i>Universal Asynchronous Receiver-Transmitter</i>
USB	<i>Universal Serial Bus</i>
USB-TMC	<i>USB Test and Measurement Class</i>
WDT	<i>Watchdog Timer</i>
XADC	<i>Xilinx Analog-to-Digital Converter</i>

LISTA DE SÍMBOLOS

AF	Fator de aceleração (<i>Acceleration Factor</i>)
E_a	Energia de ativação (eV)
k_B	Constante de Boltzmann
T_a	Temperatura ambiente do teste (temperatura do forno/câmara)
T_j	Temperatura de junção do dispositivo
T_{stress}	Temperatura de estresse aplicada durante o <i>burn-in</i>
V_{DD}	Tensão de alimentação do núcleo do circuito
θ_{ja}	Resistência térmica junção–ambiente
$P_{dynamic}$	Potência dinâmica dissipada
$P_{leakage}$	Potência associada à corrente de fuga
P_{total}	Potência total dissipada
t_{setup}	Tempo de <i>setup</i> de um flip-flop
t_{hold}	Tempo de <i>hold</i> de um flip-flop
T_{clk}	Período do clock do sistema
t_{prop}	Atraso de propagação do caminho lógico
t_{slack}	Margem de temporização (<i>slack</i>) do caminho crítico
K_p	Ganho proporcional do controlador PID
K_i	Ganho integral do controlador PID
K_d	Ganho derivativo do controlador PID
SP	<i>Setpoint</i> (referência do controlador)
PV	<i>Process Variable</i> (variável de processo)
K	Ganho estático do modelo FOPDT
τ	Constante de tempo do modelo FOPDT
θ	Tempo morto do modelo FOPDT

SUMÁRIO

1	INTRODUÇÃO	19
1.1	Motivação e Justificativa	19
1.2	Objetivos	21
1.2.1	<i>Objetivo Geral</i>	21
1.2.2	<i>Objetivos Específicos</i>	21
1.3	Estrutura do Trabalho	21
2	FUNDAMENTAÇÃO TEÓRICA	23
2.1	Mecanismos de Envelhecimento em Circuitos Nanométricos	23
2.1.1	<i>Bias Temperature Instability (BTI)</i>	23
2.1.2	<i>Hot Carrier Injection (HCI)</i>	24
2.1.3	<i>Modelagem de Aceleração: Equação de Arrhenius</i>	25
2.1.4	<i>Fenômeno de Fuga Térmica (Thermal Runaway)</i>	25
2.2	Temporização Digital e Sensores de Envelhecimento	26
2.2.1	<i>Restrições de Temporização: Setup Time e Hold Time</i>	26
2.2.2	<i>Conceito de Slack Temporal</i>	27
2.2.3	<i>Modelos de Sensores de Envelhecimento</i>	27
2.2.3.1	<i>Sensores Baseados em Osciladores</i>	27
2.2.3.2	<i>Sensores de Caminho Crítico</i>	28
2.2.4	<i>Sensor com Auto-Calibração Baseado em MMCM</i>	28
2.3	Normas Técnicas para Testes de Confiabilidade	30
2.3.1	<i>MIL-STD-883 Método 1015 (Burn-in Test)</i>	30
2.3.2	<i>JEDEC JESD22-A108 (HTOL)</i>	31
2.4	Teoria de Controle: PID e Método SIMC	31
2.4.1	<i>Comparativo de Estratégias de Controle Térmico</i>	31
2.4.2	<i>O Algoritmo de Controle PID</i>	32
2.4.3	<i>Modelagem FOPDT de Sistemas Térmicos</i>	33
2.4.4	<i>Sintonia SIMC (Skogestad)</i>	33
3	METODOLOGIA	35
3.1	Arquitetura Geral do Sistema	35
3.2	Hardware do Sistema	36

3.2.1	<i>Controlador de Temperatura: Arduino Nano 33 IoT</i>	36
3.2.2	<i>Sistema de Sensoriamento Térmico</i>	36
3.2.3	<i>Atuação de Potência: Relé de Estado Sólido (SSR)</i>	37
3.2.4	<i>Forno de Aquecimento</i>	37
3.2.5	<i>Fonte de Alimentação Programável</i>	38
3.3	Modelagem do Sistema Térmico	38
3.3.1	<i>Identificação do Sistema via FOPDT</i>	38
3.3.2	<i>Sintonia do Controlador PID via Método SIMC</i>	40
3.4	Desenvolvimento do Firmware	41
3.4.1	<i>Controle Time-Proportioning</i>	41
3.4.2	<i>Controle de Rampa (Soft-Start)</i>	41
3.4.3	<i>Protocolo de Comunicação Serial</i>	41
3.5	Arquitetura da FPGA	42
3.5.1	<i>Interface UART</i>	42
3.5.2	<i>Integração com XADC</i>	42
3.6	Software Supervisório (IHM)	43
3.6.1	<i>Arquitetura Multi-threading</i>	43
3.6.2	<i>Datalogging Sincronizado</i>	43
3.6.3	<i>Mecanismos de Segurança e Operação Fail-Safe</i>	44
3.6.4	<i>Interface Gráfica</i>	44
4	RESULTADOS E DISCUSSÃO	46
4.1	Validação do Controle Térmico PID	46
4.1.1	<i>Comparação: SIMC Teórico vs. Ganhos Após Fine-Tuning</i>	46
4.2	Comparação: Controle PID vs. Controle On-Off (Bang-Bang)	47
4.2.1	<i>Análise Temporal</i>	48
4.2.2	<i>Análise Estatística da Estabilidade Térmica</i>	48
4.3	Validação do Sistema sob Condições de Teste	49
4.3.1	<i>Impacto na Qualidade dos Dados do Sensor</i>	49
4.3.2	<i>Observação sobre Temperatura Média</i>	50
4.4	Integração do Sistema e Datalogging	51
4.4.1	<i>Estrutura dos Dados Coletados</i>	51
4.4.2	<i>Análise de Segurança</i>	52

5	CONCLUSÕES E TRABALHOS FUTUROS	53
5.1	Síntese do Trabalho	53
5.2	Avaliação dos Objetivos	53
5.2.1	<i>Objetivo Geral</i>	53
5.2.2	<i>Objetivos Específicos</i>	54
5.3	Contribuições do Trabalho	54
5.3.1	<i>Estabilidade Térmica de Precisão Industrial</i>	54
5.3.2	<i>Atenuação do Ruído Térmico nos Dados de Envelhecimento</i>	55
5.3.3	<i>Automação e Repetibilidade</i>	55
5.3.4	<i>Segurança para Operação Autônoma</i>	55
5.4	Melhorias no Sistema de Controle	56
5.5	Expansão de Funcionalidades	56
5.6	Análise Avançada de Dados	56
5.7	Considerações Finais	57
	REFERÊNCIAS	58
	 APÊNDICES	 60
	APÊNDICE A – Firmware do Controlador Térmico (Arduino Nano) . . .	61
	APÊNDICE B – Drivers e Orquestração do Ensaio (Python Workers) . . .	65
	APÊNDICE C – Interface Gráfica da IHM (Python PySide6)	69

1 INTRODUÇÃO

A confiabilidade de sistemas eletrônicos tornou-se um pilar fundamental no desenvolvimento de tecnologias modernas, especialmente naquelas destinadas a aplicações críticas como a indústria aeroespacial, automotiva, médica e de infraestrutura de defesa. Com o advento da Lei de Moore e a consequente redução das dimensões dos transistores para escalas nanométricas — como a tecnologia de 28nm utilizada nos dispositivos *Field-Programmable Gate Array* (FPGA) da família Xilinx Artix-7 — (AMD Xilinx, 2023b), a densidade de potência e os campos elétricos internos aumentaram significativamente. Esse cenário evidenciou a sensibilidade dos Circuitos Integrados (CIs) a mecanismos de envelhecimento físico, anteriormente considerados secundários, que agora representam ameaças tangíveis à vida útil e à estabilidade operacional dos dispositivos (AMOURI *et al.*, 2012; LANZIERI *et al.*, 2024).

A indústria de semicondutores tem respondido a esses desafios migrando de abordagens de projeto conservadoras e estáticas, como o *guard-banding* (margens de segurança excessivas), para estratégias dinâmicas de *Silicon Lifecycle Management* (SLM) (VARGAS; BALAKRISHNAN, 2025). O SLM pressupõe o monitoramento contínuo da saúde do chip durante sua operação em campo (*in-mission*), permitindo ajustes adaptativos de tensão e frequência para mitigar o desgaste. Para viabilizar tais estratégias, é imperativo o desenvolvimento e a calibração de sensores de envelhecimento *on-chip* capazes de quantificar a degradação do atraso de propagação (*slack*) em tempo real (GHADERI *et al.*, 2017; NOGUEIRA *et al.*, 2025).

1.1 Motivação e Justificativa

Este trabalho insere-se como uma continuação direta e um aprimoramento da pesquisa iniciada com o artigo "Auto-Tuning Aging Sensor Validated Under Burn-In, Temperature, and Voltage Variations", publicado por Nogueira *et al.* (NOGUEIRA *et al.*, 2025). O referido artigo propôs um sensor de envelhecimento inovador, baseado em deslocamento de fase dinâmico de *clocks*, capaz de medir a erosão do *slack* temporal em caminhos críticos digitais de forma autônoma, sem necessidade de recalibrações manuais.

A principal contribuição daquele trabalho foi demonstrar que sensores *on-chip* podem rastrear com precisão a degradação causada por mecanismos como *Bias Temperature Instability* (BTI) e *Hot Carrier Injection* (HCI), em tempo real, oferecendo uma alternativa superior ao *guard-banding* tradicional (AMOURI *et al.*, 2014; LANZIERI *et al.*, 2024; NOGUEIRA *et al.*,

2025). No entanto, durante a validação experimental desse sensor, foram identificadas limitações significativas na infraestrutura de testes utilizada, que comprometiam a precisão e a conformidade normativa dos resultados obtidos.

A validação desses sensores de envelhecimento e a caracterização de falhas em CIs exigem a realização de testes de vida acelerada (*Accelerated Life Testing (ALT)*), popularmente conhecidos como *burn-in*. Esses testes submetem o DUT a condições extremas de temperatura e tensão por períodos prolongados, visando simular anos de operação em poucas semanas, baseando-se em modelos físicos como a Equação de Arrhenius (JEDEC Solid State Technology Association, 2022; U.S. Department of Defense, 2016).

No experimento original, a equipe utilizou um forno odontológico para secagem e uma fonte de alimentação de bancada com automatização parcialmente integrada. Uma análise crítica pós-experimento revelou que esse aparato não atendia aos rigorosos requisitos de estabilidade exigidos pelas normas industriais e militares, especificamente:

- **Instabilidade Térmica:** O controle *on-off* (termostático) do forno gerava oscilações de temperatura ("dente de serra") superiores a $\pm 5^{\circ}\text{C}$. A norma MIL-STD-883, Método 1015, exige uma estabilidade de $\pm 3^{\circ}\text{C}$ (U.S. Department of Defense, 2016). Tais flutuações introduzem ruído nos dados de envelhecimento, dificultando a distinção entre a degradação permanente dos transistores e variações temporárias de desempenho induzidas pela temperatura.
- **Choque Térmico:** A ausência de controle de rampa de aquecimento expunha o encapsulamento da FPGA a estresses mecânicos súbitos, elevando o risco de falhas físicas e estruturais não relacionadas ao aumento de atraso nos caminhos críticos do circuito.
- **Dessincronização de Dados:** A coleta de dados do sensor (via *Joint Test Action Group (JTAG)*/*Universal Asynchronous Receiver-Transmitter (UART)* da FPGA) e dos parâmetros de temperatura e tensão do sistema eram realizados por sistemas integrados, mas o dado de temperatura do forno era inacessível, pelo uso de um termostato analógico, dificultando a correlação causal entre estresse e degradação.

Essas limitações motivaram o desenvolvimento de uma plataforma integrada, que pudesse não apenas superar os problemas identificados, mas também estabelecer um padrão acessível para pesquisas futuras em confiabilidade de semicondutores. A necessidade de um sistema que concilie baixo custo, precisão industrial e automação completa constitui a justificativa central deste trabalho.

1.2 Objetivos

1.2.1 *Objetivo Geral*

O objetivo geral deste Trabalho de Conclusão de Curso é projetar, implementar e validar uma plataforma automatizada de testes de confiabilidade para FPGAs, capaz de controlar e fornecer temperatura e tensão de forma estável, integrando hardware de baixo custo e software de supervisão com medidas de segurança automáticas.

1.2.2 *Objetivos Específicos*

Os objetivos específicos são:

1. Desenvolver um sistema de controle térmico em malha fechada utilizando um microcontrolador Arduino Nano 33 IoT, implementando um PID digital sintonizado pelo método SIMC.
2. Projetar e montar a interface de potência utilizando *Solid State Relay* (SSR) e *Negative Temperature Coefficient* (NTC) ratiométricos, visando substituir o controle termostático mecânico do forno.
3. Desenvolver uma aplicação supervisória em Python com arquitetura *multi-threading*, capaz de orquestrar o forno, a fonte de alimentação programável e a leitura do DUT simultaneamente.
4. Implementar rotinas de segurança e perfis de rampa de temperatura (*ramp rate*) para proteger o dispositivo contra choques térmicos e *overshoot*.
5. Validar o sistema comparando sua estabilidade térmica e capacidade de automação com os requisitos das normas *Joint Electron Device Engineering Council* (JEDEC) JESD22-A108 e MIL-STD-883 (JEDEC Solid State Technology Association, 2022; U.S. Department of Defense, 2016).

1.3 Estrutura do Trabalho

Este documento está organizado em cinco capítulos. O **Capítulo 2** apresenta a revisão bibliográfica necessária para a compreensão do projeto, abordando os mecanismos de envelhecimento em circuitos nanométricos (BTI, HCI), os testes de envelhecimento acelerado e suas normas regulatórias (MIL-STD-883, JEDEC), os fundamentos de circuitos síncronos e

sensores de envelhecimento, e a teoria de controle aplicada a sistemas térmicos.

O **Capítulo 3** detalha a metodologia empregada no desenvolvimento do sistema, descrevendo a arquitetura de hardware (microcontrolador, sensores, atuadores), o desenvolvimento do firmware de controle PID, a arquitetura da FPGA com o sensor de auto-tuning e a aplicação supervisória *multi-threading* desenvolvida em Python.

O **Capítulo 4** apresenta e discute os resultados obtidos, incluindo a modelagem do sistema térmico via FOPDT (*First Order Plus Dead Time*), a sintonia e validação do controlador PID, a comparação estatística entre o controle *on-off* convencional e o sistema proposto, a análise de conformidade com as normas técnicas e uma discussão sobre as limitações identificadas.

Por fim, o **Capítulo 5** sintetiza as principais contribuições do trabalho, avalia o cumprimento dos objetivos propostos e apresenta sugestões para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo estabelece os conceitos necessários para a compreensão do projeto, abrangendo a física da confiabilidade de semicondutores, os conceitos de temporização em circuitos síncronos, os sensores de envelhecimento *on-chip*, as normas técnicas aplicáveis e a teoria de controle utilizada no desenvolvimento do sistema.

2.1 Mecanismos de Envelhecimento em Circuitos Nanométricos

À medida que as tecnologias implementadas na fabricação de Circuitos Integrados (CIs) evoluem para nós tecnológicos cada vez menores, a variabilidade nos parâmetros elétricos dos transistores, como tensão de limiar e mobilidade de cargas, torna-se mais proeminente, comprometendo a confiabilidade e robustez dos dispositivos (AMOURI *et al.*, 2012). Em tecnologias de processo abaixo de 90nm, como o processo de 28nm utilizado nas FPGAs da família Xilinx 7 Series (AMD Xilinx, 2023b), os mecanismos dinâmicos de degradação tornam-se predominantes (LANZIERI *et al.*, 2024).

Com a redução contínua das dimensões dos transistores, os efeitos de envelhecimento tornam-se progressivamente mais problemáticos. O aumento das densidades de potência e a redução das margens de tensão contribuem para a degradação acelerada através dos mecanismos de BTI e HCI. Além disso, essa miniaturização é acompanhada por um aumento na sensibilidade às variações de processo, tensão de alimentação e temperatura (*Process, Voltage, and Temperature (PVT)*) (NOGUEIRA *et al.*, 2025; VALDES-PENA *et al.*, 2013).

Em aplicações espaciais, a *Total Ionizing Dose* (TID), ou radiação ionizante total, representa uma preocupação adicional para projetistas de CIs, pois acelera o envelhecimento e aumenta o tempo de resposta do circuito devido a deslocamentos na tensão de limiar e aumento de correntes de fuga. Em estados avançados de envelhecimento, pode ocorrer dessincronização, levando primeiro a falhas transientes e, eventualmente, a falhas permanentes no fim da vida útil (BENFICA *et al.*, 2016; NOGUEIRA *et al.*, 2025).

2.1.1 Bias Temperature Instability (BTI)

A Instabilidade de Temperatura e Polarização (BTI) é um dos principais mecanismos de falha em dispositivos *Complementary Metal-Oxide-Semiconductor* (CMOS) modernos (AMOURI *et al.*, 2012; LANZIERI *et al.*, 2024). O fenômeno ocorre quando os transistores são

submetidos a altas temperaturas e a um campo elétrico constante na porta (*gate*).

O **Negative Bias Temperature Instability** (*Negative Bias Temperature Instability* (**NBTI**)) afeta transistores PMOS quando polarizados negativamente ($V_{gs} < 0$). Fisicamente, o estresse causa a dissociação das ligações passivadas de Silício-Hidrogênio ($Si-H$) na interface entre o canal de silício e o dielétrico de porta (SiO_2). A reação gera “armadilhas de interface” (N_{it}) e cargas fixas positivas no óxido. O efeito macroscópico é o aumento do valor absoluto da tensão de limiar ($|V_{th}|$) do transistor (AMOURI *et al.*, 2014).

A taxa de degradação por BTI pode ser modelada pela relação:

$$\Delta V_{th}(t) = A \cdot \left(\frac{V_{DD}}{V_{DD,nom}} \right)^n \cdot t^m \cdot e^{-E_a/k_B T} \quad (2.1)$$

onde $n \approx 2.5-4$, $m \approx 0.16-0.25$ (regime difusivo), e $E_a \approx 0.1-0.15$ eV. Uma característica importante do BTI é que ele apresenta *recovery* parcial quando o transistor é desligado, recuperando tipicamente 10–30% da degradação (AMOURI *et al.*, 2014).

Conforme a equação da corrente de dreno em saturação:

$$I_{ds} \propto (V_{gs} - V_{th})^2 \quad (2.2)$$

Um aumento em V_{th} reduz a corrente de condução (I_{ds}), o que, por sua vez, aumenta o tempo de carga e descarga das capacitâncias parasitas, tornando a porta lógica mais lenta.

2.1.2 Hot Carrier Injection (HCI)

A Injeção de Portadores Quentes (HCI) ocorre quando portadores de carga (elétrons ou lacunas) ganham energia cinética excessiva ao serem acelerados por altos campos elétricos laterais no canal do transistor, tipicamente próximos à região do dreno (LANZIERI *et al.*, 2024). Se a energia desses portadores exceder a barreira de potencial da interface $Si-SiO_2$ (aproximadamente 3.2 eV para elétrons), eles podem ser injetados no óxido de porta, criando defeitos e aprisionando cargas.

Diferentemente do BTI, que é um fenômeno estático relacionado à polarização contínua, o HCI é dependente da atividade de comutação do circuito. Portanto, circuitos com alta taxa de chaveamento são mais suscetíveis a este mecanismo de degradação (GHADERI *et*

al., 2017). Uma característica distintiva do HCI é que, ao contrário do BTI, ele causa degradação permanente sem recuperação (*recovery*).

2.1.3 Modelagem de Aceleração: Equação de Arrhenius

Para estimar a vida útil de um componente em um tempo hábil, utilizam-se testes acelerados. A relação entre a temperatura e a taxa de reação química que governa a degradação (como a quebra de ligações *Si-H*) é descrita pela Lei de Arrhenius (JEDEC Solid State Technology Association, 2022; U.S. Department of Defense, 2016):

$$AF = \exp \left(\frac{E_a}{k_B} \left(\frac{1}{T_{\text{use}}} - \frac{1}{T_{\text{stress}}} \right) \right) \quad (2.3)$$

onde AF é o Fator de Aceleração, E_a é a energia de ativação do mecanismo de falha (tipicamente 0.7 eV para mecanismos de silício), k_B é a constante de Boltzmann (8.617×10^{-5} eV/K) e T é a temperatura em Kelvin.

Esta equação demonstra a sensibilidade exponencial do teste à temperatura. Uma instabilidade de poucos graus no forno altera drasticamente o fator de aceleração AF , invalidando a precisão dos cálculos de vida útil. Por exemplo, para $E_a = 0.7$ eV, uma variação de 5°C na temperatura de estresse (de 125°C para 130°C) resulta em um aumento de aproximadamente 15% no fator de aceleração. Por isso, o controle preciso da temperatura T_{stress} é o requisito mais crítico de um sistema de *burn-in*.

2.1.4 Fenômeno de Fuga Térmica (Thermal Runaway)

A execução de testes de *burn-in* em tecnologias nanométricas introduz um risco crítico de instabilidade térmica (VASSIGHI *et al.*, 2004; BAI *et al.*, 2011). Diferentemente de tecnologias antigas, onde a potência dinâmica dominava, em nós avançados a potência estática (P_{leakage}) torna-se significativa e fortemente dependente da temperatura.

A relação de *feedback* positivo que leva à fuga térmica pode ser descrita pela interação entre a temperatura da junção (T_j) e a corrente de fuga (I_{leak}). A temperatura da junção é dada pelo balanço térmico (VASSIGHI *et al.*, 2004):

$$T_j = T_a + (P_{\text{dynamic}} + P_{\text{leakage}}(T_j, V_{DD})) \cdot \theta_{ja} \quad (2.4)$$

onde T_a é a temperatura ambiente (controlada pelo forno), θ_{ja} é a resistência térmica junção-ambiente, e $P_{leakage}$ cresce exponencialmente com a temperatura. A condição de *Thermal Runaway* ocorre quando a taxa de geração de calor excede a capacidade de dissipação do encapsulamento, ou seja, quando a derivada da potência gerada em relação à temperatura supera o inverso da resistência térmica (BAI *et al.*, 2011):

$$\frac{dP_{total}}{dT_j} > \frac{1}{\theta_{ja}} \quad (2.5)$$

Com o escalonamento da tecnologia CMOS, a corrente de fuga *off-state* tornou-se o problema mais sério, com a proporção entre corrente de fuga e corrente total aumentando continuamente (BAI *et al.*, 2011). Além disso, diferenças nos parâmetros dos dispositivos semicondutores, causadas pelo próprio processo de fabricação em silício, resultam em temperaturas de junção desiguais mesmo sob condições idênticas de teste (VASSIGHI *et al.*, 2004).

O sistema de controle proposto neste trabalho atua na regulação dinâmica de T_a para compensar as variações em P_{total} , garantindo que T_j permaneça em uma região estável de operação, evitando danos catastróficos ao dispositivo sob teste (DUT).

2.2 Temporização Digital e Sensores de Envelhecimento

Esta seção apresenta os conceitos fundamentais de temporização em circuitos digitais síncronos e as técnicas de monitoramento *on-chip* do envelhecimento, que constituem a base teórica para compreender a relevância do controle térmico preciso na validação de sensores de degradação.

2.2.1 Restrições de Temporização: Setup Time e Hold Time

Em circuitos digitais síncronos, a operação correta depende do cumprimento de duas restrições temporais fundamentais associadas aos elementos de memória (flip-flops) (GHADERI *et al.*, 2017):

- **Tempo de Setup (t_{setup}):** Intervalo mínimo durante o qual o sinal de dados deve permanecer estável *antes* da borda ativa do clock. Garante que o dado seja corretamente amostrado.
- **Tempo de Hold (t_{hold}):** Intervalo mínimo durante o qual o sinal de dados deve permanecer estável *após* a borda ativa do clock. Evita que variações no dado corrompam o valor registrado.

A violação dessas restrições pode levar o flip-flop a um estado metaestável, onde a saída permanece indefinida por um período indeterminado, podendo propagar erros pelo circuito.

2.2.2 Conceito de Slack Temporal

O *slack* temporal representa a margem de segurança disponível em um caminho lógico para garantir operação correta. É definido como a diferença entre o período do clock e o atraso de propagação do caminho crítico, considerando os tempos de setup (NOGUEIRA *et al.*, 2025; VARGAS *et al.*, 2024):

$$t_{slack} = T_{clk} - (t_{prop} + t_{setup}) \quad (2.6)$$

onde T_{clk} é o período do clock do sistema, t_{prop} é o atraso de propagação do caminho combinacional, e t_{setup} é o tempo de setup do flip-flop de destino.

Um *slack* positivo indica que o circuito opera dentro das margens de temporização. À medida que o circuito envelhece, o atraso de propagação t_{prop} aumenta devido ao incremento em V_{th} causado por BTI e HCI, reduzindo progressivamente o *slack* disponível. Quando o *slack* atinge zero ou se torna negativo, ocorrem violações de temporização que resultam em falhas funcionais (NOGUEIRA *et al.*, 2025).

2.2.3 Modelos de Sensores de Envelhecimento

Sensores *on-chip* para monitoramento de envelhecimento dividem-se em duas categorias principais (LANZIERI *et al.*, 2024):

2.2.3.1 Sensores Baseados em Osciladores

Osciladores em anel (*Ring Oscillators* - ROs) são estruturas compostas por um número ímpar de inversores em cascata, cuja frequência de oscilação diminui à medida que os transistores envelhecem (AHMED *et al.*, 2021). A vantagem desta abordagem é sua simplicidade de implementação e baixo *overhead* de área. A principal desvantagem é que a frequência do oscilador não correlaciona diretamente com o atraso de caminhos críticos reais do circuito monitorado, podendo subestimar ou superestimar a degradação efetiva.

2.2.3.2 Sensores de Caminho Crítico

Sensores de caminho crítico monitoram diretamente o atraso de caminhos lógicos específicos através de flip-flops operando com clocks deslocados em fase (GHADERI *et al.*, 2017; VARGAS *et al.*, 2024). A arquitetura desses sensores pode ser representada pela figura 1 abaixo:

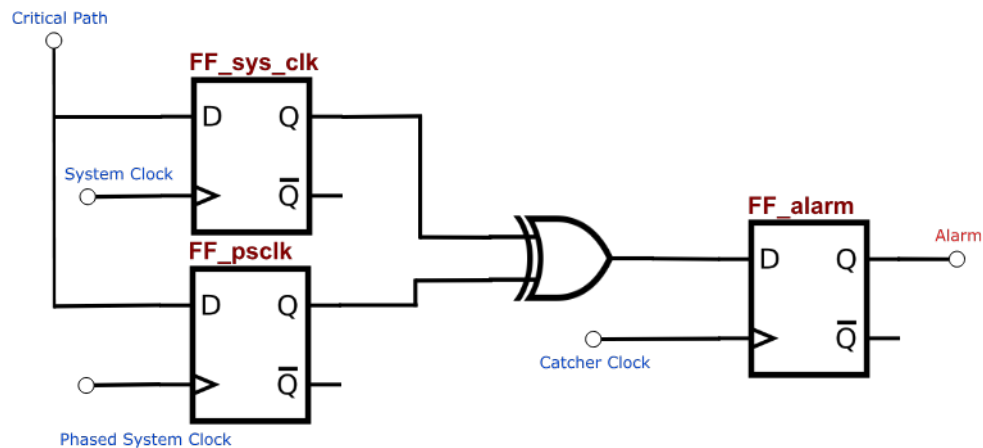


Figura 1 – Representação da arquitetura de sensores de caminho crítico baseados em flip-flops. Fonte: (NOGUEIRA *et al.*, 2025).

Estes sensores consistem em dois flip-flops recebendo o sinal do caminho crítico em suas entradas, e dois clocks de mesmo período, porém levemente faseados um em relação ao outro. A saída desses flip-flops entram em um comparador XOR, que envia seu sinal de saída para um terceiro flip-flop, com um clock atrasado em relação aos outros dois.

Quando o atraso do caminho monitorado excede o deslocamento de fase aplicado, os flip-flops capturam valores distintos, ativando o alarme. Esta abordagem permite correlação direta com o desempenho real do circuito, mas requer recursos de gerenciamento de clock que são escassos em FPGAs e ASICs.

As figuras 2 e 3 ilustram conceitualmente essa relação entre o atraso do caminho crítico e o *slack* disponível. Em um circuito novo (não envelhecido), como na figura 2, o sinal estabiliza com margem suficiente antes da borda do clock. Com o envelhecimento, como na figura 3, o atraso aumenta e o *slack* diminui até que violações de *timing* ocorram.

2.2.4 Sensor com Auto-Calibração Baseado em MMCM

O trabalho de Nogueira et al. (NOGUEIRA *et al.*, 2025) propôs uma evolução da arquitetura de sensores de caminho crítico, incorporando um mecanismo de auto-calibração dinâ-

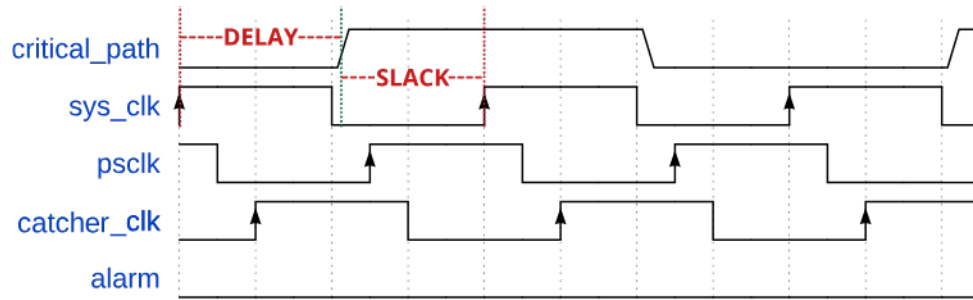


Figura 2 – Diagrama de temporização mostrando o comportamento esperado do *slack* devido em um circuito novo.

Fonte: (NOGUEIRA *et al.*, 2025).

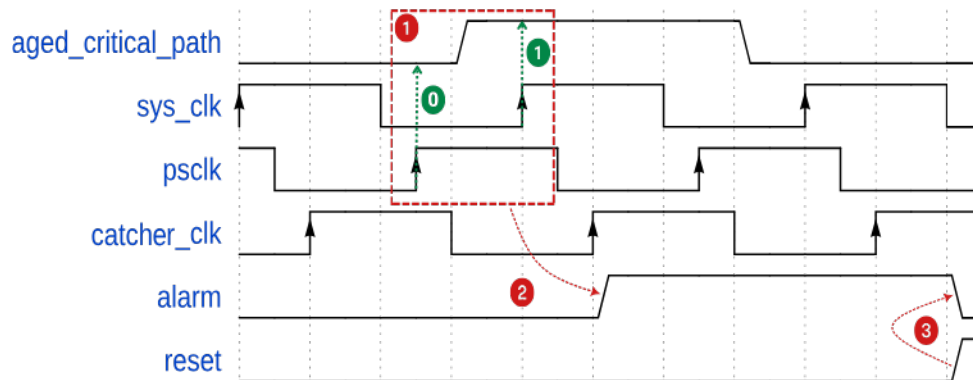


Figura 3 – Diagrama de temporização mostrando a erosão do *slack* devido ao envelhecimento do circuito.

Fonte: (NOGUEIRA *et al.*, 2025).

mica. Diferentemente das abordagens convencionais onde a calibração é realizada estaticamente durante a configuração da FPGA, este sensor utiliza o recurso de deslocamento dinâmico de fase disponível nos MMCMs (*Mixed-Mode Clock Managers*) das FPGAs Xilinx Série 7 e posteriores (AMD Xilinx, 2023a).

O procedimento de auto-calibração funciona da seguinte forma (NOGUEIRA *et al.*, 2025):

1. Inicialmente, *psclk* é alinhado com *sys_clk*
2. O controlador incrementa progressivamente a fase de *psclk*, adiantando-o em relação a *sys_clk*
3. Este processo continua até que o alarme seja ativado
4. O número de incrementos aplicados, multiplicado pela resolução de fase (19.85 ps no trabalho citado), corresponde ao valor do *slack*

Esta abordagem permite recalibração em tempo de execução sempre que uma variação no atraso do caminho crítico é detectada, habilitando o monitoramento contínuo durante a operação em missão. Este é um requisito fundamental para estratégias de SLM (NOGUEIRA *et*

al., 2025; VARGAS; BALAKRISHNAN, 2025).

A validação experimental de sensores de envelhecimento requer a indução controlada de degradação através de procedimentos de *burn-in*. A precisão dos dados obtidos depende diretamente da estabilidade das condições de estresse (NOGUEIRA *et al.*, 2025). Entretanto, oscilações térmicas durante o *burn-in* introduzem ruído nas medições do sensor, dificultando a separação entre a degradação permanente (devido ao envelhecimento) e as variações temporárias (devido à flutuação de temperatura).

2.3 Normas Técnicas para Testes de Confiabilidade

A padronização dos testes é essencial para garantir a repetibilidade e a aceitação industrial dos dados. Os procedimentos de envelhecimento acelerado utilizados na indústria seguem especificações técnicas estabelecidas por organizações militares e consórcios industriais (JEDEC Solid State Technology Association, 2022; U.S. Department of Defense, 2016; U.S. Department of Defense, 2010).

2.3.1 MIL-STD-883 Método 1015 (*Burn-in Test*)

O padrão militar MIL-STD-883 é a referência primária para testes de microeletrônica em aplicações de defesa e aeroespaciais (U.S. Department of Defense, 2016). O Método 1015 especifica os requisitos para o teste de *Burn-in*, que visa precipitar falhas de mortalidade infantil em dispositivos semicondutores antes de sua aplicação em campo.

Os principais requisitos do método incluem:

- A câmara de teste deve ser capaz de manter a temperatura especificada dentro de uma tolerância de $\pm 3^{\circ}\text{C}$ (ou $\pm 3\%$) em qualquer ponto da área de trabalho
- Monitoramento contínuo para evitar fugas térmicas (*thermal runaway*) em dispositivos de alta potência
- Temperatura típica de teste: 125°C
- Duração típica: 160 horas

A falha do sistema manual anterior em manter essa janela de $\pm 3^{\circ}\text{C}$ de forma consistente ao longo de testes prolongados é uma das principais justificativas para a automação proposta neste trabalho.

2.3.2 JEDEC JESD22-A108 (HTOL)

A norma JEDEC JESD22-A108 define o teste de *High Temperature Operating Life* (HTOL), amplamente utilizado na indústria de semicondutores comerciais (JEDEC Solid State Technology Association, 2022). O objetivo é estressar o dispositivo sob condições de polarização dinâmica para revelar mecanismos de falha dependentes do tempo.

Requisitos principais da norma JEDEC:

- A temperatura da junção (T_j) deve ser controlada e conhecida
- Tolerância de temperatura: $\pm 2^\circ\text{C}$ para garantir validade estatística
- Tensão de estresse: tipicamente $V_{DD} + 10\%$ a $V_{DD} + 50\%$
- Monitoramento de corrente para detecção de falhas

A norma também especifica condições para diferentes níveis de aceleração, como teste a 85°C , 105°C e 125°C , permitindo o cálculo de energias de ativação através de testes em múltiplas temperaturas.

2.4 Teoria de Controle: PID e Método SIMC

Para substituir o controle termostático (*on-off*), que inerentemente gera oscilações em torno do *setpoint*, utiliza-se um controlador Proporcional-Integral-Derivativo (PID) (XU *et al.*, 2024). Esta seção apresenta os fundamentos teóricos do controle PID e o método de sintonia empregado.

2.4.1 Comparativo de Estratégias de Controle Térmico

A escolha do algoritmo de controle impacta diretamente a validade dos dados de envelhecimento. A Tabela 1 resume as principais características das estratégias de controle aplicáveis a sistemas térmicos.

Controladores do tipo *On-Off* (ou *bang-bang*), comumente encontrados em estufas laboratoriais genéricas, introduzem oscilações de baixa frequência em torno do *setpoint* devido ao efeito de histerese (BAI *et al.*, 2011). Embora de baixo custo, esse método resulta em um perfil de temperatura $T(t)$ caracterizado por erro cíclico:

$$T_{erro} = T_{set} \pm \Delta T_{histerese} \quad (2.7)$$

Tabela 1 – Comparativo entre estratégias de controle térmico

Característica	On-Off	PID	Controle Avançado
Complexidade de implementação	Baixa	Média	Alta
Erro de regime permanente	Alto	Baixo	Zero
Oscilação em torno do setpoint	Alta	Baixa	Mínima
Custo computacional	Mínimo	Baixo	Alto
Adequação para burn-in	Inadequado	Adequado	Adequado

Fonte: Elaborado pelo autor.

Para testes de confiabilidade regidos pelas normas MIL-STD-883 ($\pm 3^\circ\text{C}$) e JEDEC ($\pm 2^\circ\text{C}$), essas oscilações introduzem ruído nos dados de degradação, comprometendo a correlação entre estresse e resposta do dispositivo.

2.4.2 O Algoritmo de Controle PID

O controlador PID calcula a ação de controle $u(t)$ baseada no erro $e(t) = SP - PV$, onde SP é o *setpoint* (temperatura desejada) e PV é a variável de processo (temperatura medida) (XU *et al.*, 2024):

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.8)$$

As três componentes do controlador possuem funções específicas:

- **Ação Proporcional (K_p):** Produz uma saída proporcional ao erro presente. Aumentar K_p acelera a resposta do sistema, mas valores excessivos podem causar instabilidade e oscilações.
- **Ação Integral (K_i):** Elimina o erro de regime permanente (*offset*) acumulando o erro ao longo do tempo. É fundamental em fornos onde há perda constante de calor para o ambiente. Valores excessivos de K_i podem causar *overshoot* e oscilações de baixa frequência.
- **Ação Derivativa (K_d):** Reage à taxa de variação do erro, antecipando tendências. Amortece o *overshoot* durante transições de aquecimento. Valores excessivos tornam o sistema sensível a ruído de medição.

Para implementação em microcontroladores, utiliza-se a forma discreta do algoritmo PID (forma de velocidade):

$$\Delta u(k) = K_p \cdot \Delta e(k) + K_i \cdot e(k) + K_d \cdot [\Delta e(k) - \Delta e(k-1)] \quad (2.9)$$

onde $\Delta e(k) = e(k) - e(k-1)$ é a variação do erro entre amostras consecutivas.

2.4.3 Modelagem FOPDT de Sistemas Térmicos

Sistemas térmicos como fornos são caracterizados por grande inércia térmica e atrasos de transporte significativos. Eles podem ser adequadamente modelados como sistemas de Primeira Ordem com Tempo Morto (FOPDT - *First Order Plus Dead Time*):

$$G(s) = \frac{K}{\tau s + 1} \cdot e^{-\theta s} \quad (2.10)$$

onde:

- K é o ganho estático do processo ($^{\circ}\text{C}/\%$ de potência)
- τ é a constante de tempo dominante (s)
- θ é o tempo morto ou atraso de transporte (s)

Os parâmetros do modelo FOPDT podem ser obtidos experimentalmente através de um teste de resposta ao degrau em malha aberta:

1. Com o sistema em regime permanente a uma temperatura inicial T_0 , aplica-se um degrau de potência (ex: de 0% para 50%)
2. Registra-se a curva de reação $T(t)$ até o novo regime permanente T_{∞}
3. Os parâmetros são extraídos da curva:
 - $K = (T_{\infty} - T_0) / \Delta u$
 - θ = tempo até a temperatura começar a variar significativamente
 - τ = tempo para atingir 63.2% da variação total (medido a partir de θ)

2.4.4 Sintonia SIMC (Skogestad)

O método SIMC oferece regras analíticas para determinar os ganhos do PID baseando-se nos parâmetros do modelo FOPDT (MORARI *et al.*, 1984). Diferentemente do método de Ziegler-Nichols, que tende a produzir respostas agressivas e com oscilações (MORARI *et al.*, 1984), o SIMC prioriza robustez e suavidade da resposta, características ideais para evitar danos ao componente eletrônico sob teste (DESAI *et al.*, 2016).

As regras SIMC para um controlador PI são:

$$K_c = \frac{1}{K} \cdot \frac{\tau}{\tau_c + \theta} \quad (2.11)$$

$$\tau_I = \min(\tau, 4(\tau_c + \theta)) \quad (2.12)$$

onde τ_c é o parâmetro de sintonia que define a velocidade de resposta desejada em malha fechada. A recomendação padrão é $\tau_c = \theta$, o que proporciona um bom compromisso entre velocidade de resposta e robustez a incertezas no modelo. Os ganhos na forma padrão são então:

$$K_p = K_c, \quad K_i = \frac{K_c}{\tau_I} \quad (2.13)$$

No presente trabalho, os parâmetros FOPDT foram obtidos experimentalmente através de um teste de degrau em malha aberta, e os ganhos iniciais do PID foram calculados pelas regras SIMC. Ajustes finos subsequentes foram realizados empiricamente para otimizar o desempenho no sistema real.

3 METODOLOGIA

Este capítulo descreve a arquitetura do sistema desenvolvido, detalhando as escolhas de hardware, a implementação do firmware de controle, a arquitetura da FPGA com o sensor de envelhecimento e o desenvolvimento do software supervisor.

3.1 Arquitetura Geral do Sistema

A plataforma foi projetada seguindo princípios de modularidade e baixo custo, utilizando exclusivamente componentes comerciais (*Commercial Off-The-Shelf (COTS)*). Esta abordagem visa a viabilização do acesso a testes de confiabilidade de alta precisão, possibilitando que outros laboratórios e ambientes acadêmicos realizem esses testes atingindo as normas exigidas para fazê-los.

A arquitetura do sistema pode ser dividida em dois subsistemas, conforme ilustrado na Figura 4:

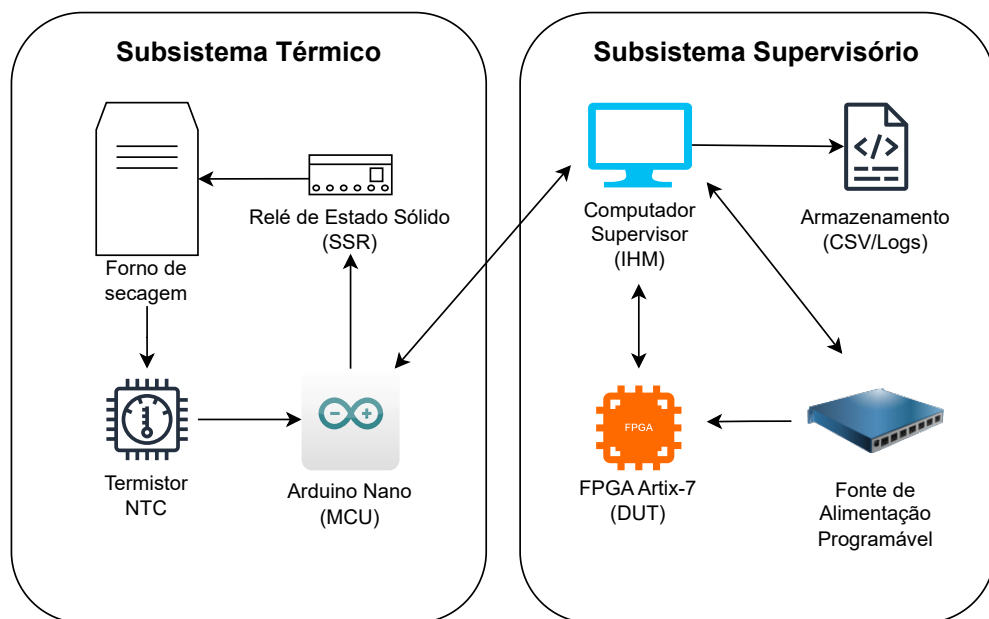


Figura 4 – Diagrama de blocos do sistema automatizado de testes de envelhecimento.

Fonte: Elaborado pelo autor.

1. **Subsistema Térmico:** Composto pelo microcontrolador Arduino Nano 33 IoT, sensor NTC e relé SSR, responsável pela execução do algoritmo PID em tempo real.
2. **Subsistema Supervisório:** Aplicação Python com interface gráfica, responsável pela orquestração do teste, coleta sincronizada de dados e visualização em tempo real.

3. **DUT:** FPGA Artix-7 com o sensor de envelhecimento implementado, incluindo interfaces UART e *Xilinx Analog-to-Digital Converter (XADC)*.

3.2 Hardware do Sistema

Especificando os dispositivos pertencentes ao subsistema térmico, é válido pontuar que a base do que foi utilizado, como o forno de secagem, são equipamentos oriundos do artigo que inspirou este trabalho, sendo adicionados, neste contexto, dispositivos adicionais para a implementação das melhorias propostas.

3.2.1 Controlador de Temperatura: Arduino Nano 33 IoT

A escolha do Arduino Nano 33 IoT como controlador central justifica-se por suas capacidades superiores em relação aos modelos clássicos de 8 bits (como o Arduino Uno), sendo fundamental para a precisão do controle térmico:

- **Processador:** SAMD21 Cortex-M0+ de 32 bits, operando a 48 MHz. Esta arquitetura permite a execução de cálculos em ponto flutuante em software (necessários para o PID) com desempenho adequado ao período de amostragem adotado.
- **Analog-to-Digital Converter (ADC) de 12-bits:** O conversor analógico-digital possui resolução configurável de 12 bits ($2^{12} = 4096$ níveis). Em uma referência de 3.3V, isso resulta em uma sensibilidade de aproximadamente 0.8 mV por passo, contra 3.22 mV dos ADCs de 10 bits padrão. Essa quadruplicação da resolução é crucial para detectar variações de temperatura inferiores a 0.1°C.
- **Conectividade:** A porta *Universal Serial Bus* (USB) nativa permite comunicação serial de alta velocidade e confiável com o PC supervisor, eliminando a latência adicional de conversores USB-Serial externos.

3.2.2 Sistema de Sensoriamento Térmico

Utilizou-se um termistor NTC (*Negative Temperature Coefficient*) de 10k Ω (coeficiente $\beta = 3950$). Para maximizar a precisão da leitura e eliminar erros provenientes de flutuações na tensão de alimentação, implementou-se uma leitura **ratiométrica**.

O circuito divisor de tensão é alimentado pela mesma tensão de referência ($V_{ref} = 3.3V$) utilizada pelo ADC. Desta forma, variações na tensão de alimentação afetam igualmente o

divisor e a referência do ADC, cancelando-se mutuamente. A resistência do termistor (R_{ntc}) é calculada pela fórmula:

$$R_{ntc} = R_{div} \cdot \left(\frac{ADC_{valor}}{ADC_{max} - ADC_{valor}} \right) \quad (3.1)$$

Onde R_{div} é um resistor de precisão de $10k\Omega$ (tolerância de 1%). A conversão para temperatura utiliza a equação Beta simplificada:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln \left(\frac{R_{ntc}}{R_0} \right) \quad (3.2)$$

Onde $T_0 = 298.15$ K (25°C) e $R_0 = 10$ k Ω são os valores de referência do termistor.

No firmware, implementou-se também uma técnica de *oversampling* e média móvel (500 amostras por leitura) para filtrar ruídos elétricos de alta frequência, melhorando estabilidade e repetibilidade da leitura do ADC.

3.2.3 Atuação de Potência: Relé de Estado Sólido (SSR)

O controle da resistência de aquecimento do forno (carga AC de 220V) é realizado por um Relé de Estado Sólido (SSR), modelo LOTI 48D25A, com capacidade de 25A.

Diferente de relés eletromecânicos, o SSR não possui partes móveis, permitindo ciclos de comutação sem desgaste físico, o que resulta em uma vida útil elevada para esse componente. Além disso, o modelo escolhido possui acionamento por *Zero-Crossing*, ou seja, ele só comuta quando a onda senoidal da rede AC passa por zero volts. Isso reduz drasticamente a geração de Interferência Eletromagnética (EMI) e surtos de tensão (*inrush current*), protegendo os circuitos de alimentação do forno e, consequentemente, o sensível circuito da FPGA que está sendo testada dentro dele.

3.2.4 Forno de Aquecimento

Utilizou-se um forno laboratorial odontológico, modelo EL 1.1 (Odontobras) com volume interno de 30 litros. O controlador termostático original (on-off) foi completamente substituído pelo sistema de controle PID desenvolvido, mantendo-se apenas a resistência de aquecimento original.

3.2.5 Fonte de Alimentação Programável

A fonte de alimentação IT6502D (ITECH) foi utilizada para fornecer tensão controlada ao núcleo da FPGA. As principais características relevantes são:

- Faixa de tensão: 0-36V com resolução de 1mV
- Faixa de corrente: 0-3A com resolução de 1mA
- Interface de comunicação: *USB Test and Measurement Class* (USB-TMC) com comandos SCPI
- Proteção contra sobrecorrente (OCP) configurável

A tensão nominal da FPGA Artix-7 (V_{CCINT}) é de 1.0V, podendo ser elevada até 1.05V para condições normais ou até 1.3V (+30%) para testes de estresse acelerado, conforme especificado pelo fabricante.

3.3 Modelagem do Sistema Térmico

Sistemas térmicos como fornos são caracterizados por grande inércia térmica e atrasos de transporte significativos. Para projetar um controlador PID adequado, é necessário primeiro obter um modelo matemático do sistema.

3.3.1 Identificação do Sistema via FOPDT

O modelo de Primeira Ordem com Tempo Morto (FOPDT - *First Order Plus Dead Time*) é amplamente utilizado para sistemas térmicos devido à sua simplicidade e capacidade de capturar as características dinâmicas essenciais do sistema (DESAI *et al.*, 2016).

Para identificar esses parâmetros da função de transferência obtida pela modelagem FOPDT, descrita pela equação 2.10, realizou-se um teste de degrau em malha aberta. O procedimento consistiu em:

1. Estabilizar o sistema na temperatura ambiente ($T_{inicial} = 25.21^{\circ}\text{C}$)
2. Aplicar um degrau de 50% na saída do controlador (potência constante)
3. Registrar a resposta de temperatura até a estabilização

O primeiro passo para a modelagem FOPDT do sistema foi feito através de um teste de degrau em malha aberta (VASSIGHI *et al.*, 2004; BAI *et al.*, 2011). A Figura 5 apresenta a resposta temporal do forno quando submetido a 50% de potência constante a partir da temperatura ambiente.

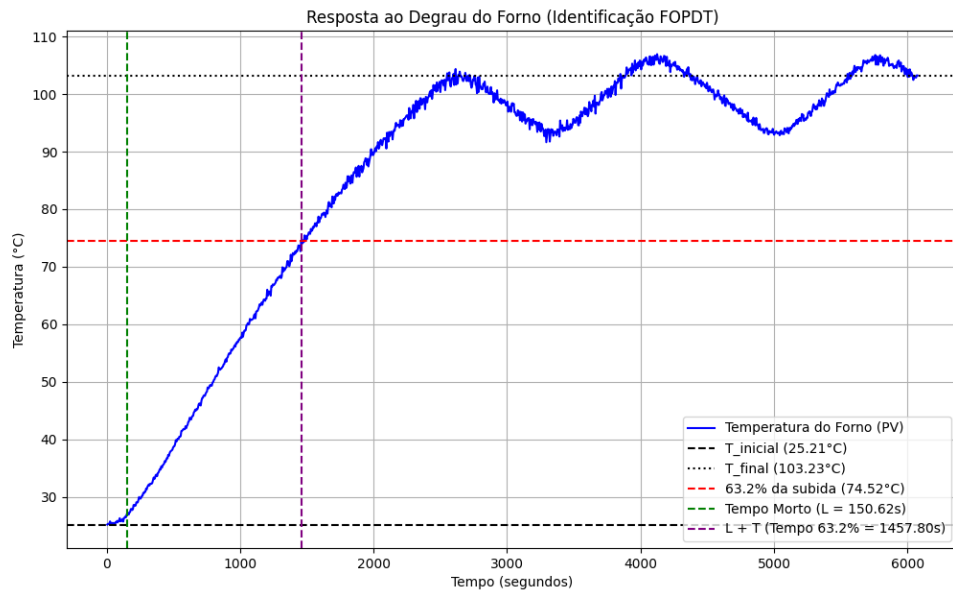


Figura 5 – Resposta ao degrau do forno para identificação dos parâmetros FOPDT. A linha verde tracejada indica o tempo morto, a linha vermelha indica 63.2% da subida, e a linha roxa indica o tempo em que essa temperatura foi atingida.

Tabela 2 – Parâmetros FOPDT identificados experimentalmente

Parâmetro	Símbolo	Valor
Temperatura inicial	$T_{inicial}$	25.21 °C
Temperatura final	T_{final}	103.23 °C
Variação de temperatura	ΔT	78.02 °C
63.2% da subida	$T_{63.2\%}$	74.52 °C
Tempo morto	θ	150.62 s
Constante de tempo	τ	1307.18 s
Ganho estático	K	1.56 °C/%

Fonte: Elaborado pelo autor a partir de dados experimentais.

Como resultado das análises do gráfico da resposta ao degrau do forno e dos parâmetros extraídos dele, representados na Tabela 2, podemos caracterizar o nosso modelo FOPDT da seguinte forma:

- Ganho estático: $K = \frac{\Delta T}{\Delta u} = \frac{78.02^\circ\text{C}}{50\%} = 1.56 \text{ }^\circ\text{C}/\%$
- Tempo morto: $\theta = 150.62 \text{ s}$
- Constante de tempo: $\tau = 1457.80 - 150.62 = 1307.18 \text{ s}$

$$G(s) = \frac{1.56}{1307.2s + 1} e^{-150.6s} \quad (3.3)$$

Este modelo captura adequadamente as características dominantes do sistema: o longo tempo morto (≈ 2.5 minutos) devido à inércia térmica da massa do forno, e a grande

constante de tempo (≈ 22 minutos) característica de processos térmicos de baixa potência.

3.3.2 Sintonia do Controlador PID via Método SIMC

O método SIMC foi escolhido para a sintonia do controlador PID por oferecer um compromisso equilibrado entre desempenho e robustez (MORARI *et al.*, 1984), sendo especialmente adequado para sistemas com grande tempo morto como fornos.

A partir das regras SIMC para um controlador PID, definidas pelas equações 2.11 e 2.12, e dos valores obtidos pela modelagem FOPDT, foi realizada a sintonia do controlador. Adotando, portanto, $\tau_c = \theta = 150.6$ s, obtém-se:

$$K_c = \frac{1}{1.56} \cdot \frac{1307.2}{150.6 + 150.6} = 2.78 \quad (3.4)$$

$$\tau_I = \min(1307.2, 4 \times 301.2) = 1204.8 \text{ s} \quad (3.5)$$

Convertendo para ganho integral:

$$K_i = \frac{K_c}{\tau_I} = \frac{2.78}{1204.8} = 0.00231 \quad (3.6)$$

Após validação experimental, os ganhos foram refinados empiricamente (*fine-tuning*) para otimizar o desempenho em regime permanente. Os ganhos finais utilizados foram:

Tabela 3 – Comparativo entre ganhos SIMC teóricos e após *fine-tuning*

Ganho	SIMC Teórico	Após <i>Fine-Tuning</i>	Unidade
K_p	2.78	2.78	-
K_i	0.00231	0.00106	s^{-1}
K_d	0	5.0	s

Fonte: Elaborado pelo autor.

A redução do ganho integral e a adição de uma componente derivativa visaram minimizar o *overshoot* durante a rampa de aquecimento e reduzir as oscilações em regime permanente.

3.4 Desenvolvimento do Firmware

O firmware foi desenvolvido em C++ utilizando a biblioteca ArduPID. A lógica de controle opera em um laço determinístico com período de amostragem de 1 segundo.

3.4.1 Controle *Time-Proportioning*

Como o SSR é um dispositivo digital (Ligado/Desligado), não é possível variar a tensão entregue à resistência de forma contínua. Implementou-se, portanto, o algoritmo de *Time-Proportioning Control* (TPC), que converte a saída contínua do PID (0-100%) em modulação por largura de pulso de baixa frequência.

Definiu-se uma janela de tempo fixa ($T_{window} = 5000$ ms). A saída do PID determina quanto tempo o SSR permanece ligado dentro dessa janela:

$$t_{on} = \frac{PID_{output}}{100} \times T_{window} \quad (3.7)$$

Por exemplo, para 20% de potência, o SSR liga por 1 segundo e desliga por 4 segundos. Devido à grande inércia térmica do forno (constante de tempo de ≈ 22 minutos), esses pulsos são integrados termicamente, resultando em uma temperatura efetivamente constante.

3.4.2 Controle de Rampa (*Soft-Start*)

Para atender aos requisitos das normas JEDEC e evitar choque térmico no encapsulamento da FPGA, implementou-se a função `updateSetpointRamp`. O usuário define um *Setpoint* final (ex: 125°C), mas o PID persegue um *Setpoint* intermediário que incrementa a uma taxa fixa (reconfigurável via protocolo UART).

Este mecanismo garante uma curva de aquecimento suave e linear, prevenindo:

- Danos mecânicos ao *die* da FPGA por expansão térmica rápida
- Delaminação do encapsulamento *Ball Grid Array* (BGA)
- Fadiga de soldas por estresse térmico repetitivo

3.4.3 Protocolo de Comunicação Serial

O firmware implementa um protocolo de comunicação textual simples via UART (115200 baud). Os principais comandos são:

- **GET_DATA:** Retorna temperatura atual, setpoint e saída do PID
- **SET_SP:XXX.X:** Define novo setpoint em °C
- **SET_RAMP:X.X:** Define taxa de rampa em °C/s
- **HEARTBEAT:** Sinal de *keep-alive* para o mecanismo de segurança

3.5 Arquitetura da FPGA

O sensor de envelhecimento implementado na FPGA Artix-7 (placa Nexys 4 DDR) é baseado na arquitetura proposta por Nogueira *et al.* (NOGUEIRA *et al.*, 2025), com modificações para integração com o sistema supervisor.

O funcionamento dos sensores permanece o mesmo, conforme citado na seção 2.2.4, com o repasse do valor de *slack* detectado em um dado momento mediante a recepção de um sinal recebido no barramento UART da FPGA. Esse protocolo de comunicação serial foi expandido para estar em conformidade com o formato de pacotes exigidos pela aplicação de monitoramento.

3.5.1 Interface UART

Para comunicação com o sistema supervisor, implementou-se um módulo UART (115200 baud, 8N1) que transmite periodicamente:

- Contagem de incrementos de fase (representando o *slack*)
- Flag de alarme
- Timestamp interno

3.5.2 Integração com XADC

O *Xilinx Analog-to-Digital Converter* (XADC) integrado à Artix-7 foi utilizado para monitorar:

- **Temperatura da FPGA:** Sensor de temperatura interno, com resolução de 0.49°C
- **Tensão V_{CCINT} :** Tensão do núcleo, permitindo verificar a alimentação fornecida pela fonte

Estes dados são transmitidos junto com a leitura do sensor, permitindo correlacionar diretamente as condições de estresse com a degradação observada.

3.6 Software Supervisor (IHM)

A aplicação desktop foi desenvolvida em Python utilizando o framework gráfico Qt (PySide6). A arquitetura do software foi desenhada para garantir robustez e responsividade, utilizando o padrão de projeto **Worker-Signal-Slot**.

3.6.1 Arquitetura Multi-threading

Para evitar o congelamento da interface gráfica durante operações de I/O (comunicação serial), cada subsistema de hardware é controlado por uma *thread* independente:

- **ArduinoWorker:** Gerencia a comunicação serial com o microcontrolador, enviando setpoints e lendo a temperatura atual e a saída do PID.
- **PSUWorker:** Controla a fonte de alimentação programável via comandos SCPI, ajustando a tensão V_{DD} do núcleo da FPGA e monitorando a corrente consumida.
- **DUTWorker:** Comunica-se com a FPGA via UART para ler os dados do sensor de envelhecimento (contagem de *slack*), temperatura do *die* e tensão V_{CCINT} .
- **TestSequencer:** É a *thread* coordenadora. Ela orquestra o início e o fim do teste, sincroniza a coleta de dados de todos os *workers*, aplica *timestamps* e grava os dados em arquivo *Comma-Separated Values* (CSV).

3.6.2 Datalogging Sincronizado

Uma das principais falhas do sistema utilizado no artigo anterior era a dessincronização dos dados. O TestSequencer coleta dados de todas as fontes a cada intervalo de tempo configurado (1 segundo por padrão) e consolida em uma única linha de registro com os seguintes campos:

- `time_sec`: Timestamp em segundos desde o início do teste
- `oven_temp_c`: Temperatura do forno (sensor NTC)
- `oven_setpoint_c`: Setpoint atual do controlador
- `oven_output_pct`: Saída do PID em porcentagem
- `psu_voltage_v`: Tensão programada na fonte
- `psu_current_a`: Corrente consumida pelo DUT
- `dut_temp_c`: Temperatura do *die* (XADC)
- `dut_slack`: Contagem de incrementos de fase (proxy do *slack*)

- `dut_volt`: Tensão V_{CCINT} medida pelo XADC

Esta sincronização garante que cada medida de degradação do sensor esteja perfeitamente correlacionada com a temperatura e a tensão exatas daquele instante, permitindo análises causais precisas.

3.6.3 Mecanismos de Segurança e Operação Fail-Safe

Considerando a natureza de longa duração dos testes de vida acelerada (até 1000 horas conforme JEDEC), o sistema foi projetado para operação autônoma (*unattended operation*). Foram implementadas camadas redundantes de segurança:

1. **Watchdog Timer (WDT) de Hardware:** O microcontrolador configura um WDT interno. Caso o loop de controle trave por interferência eletromagnética ou *bug* de software, o sistema reinicia automaticamente em estado seguro (aquecedor desligado).
2. **Timeout de Comunicação (Heartbeat):** O firmware espera um comando de *keep-alive* do supervisor a cada 5 segundos. Na ausência deste sinal (indicando falha no PC, desconexão USB ou travamento do software), o controlador força a saída do PWM para 0%, desligando o aquecimento.
3. **Limites Absolutos (Hard Limits):** Uma verificação condicional rígida no firmware desativa o atuador imediatamente se $T_{forno} > 130^{\circ}\text{C}$ ou $T_{DUT} > 140^{\circ}\text{C}$, independentemente do *setpoint* recebido.
4. **Proteção de Sobrecorrente:** A fonte de alimentação é configurada com limite de corrente (OCP) de 1.5A. Caso a FPGA entre em *latch-up* ou curto-circuito, a fonte desliga automaticamente.

3.6.4 Interface Gráfica

A interface gráfica permite ao operador:

- Visualizar em tempo real a temperatura do forno, temperatura do DUT e leitura do sensor
- Configurar setpoints de temperatura e tensão
- Definir perfis de rampa de aquecimento
- Iniciar, pausar e abortar testes
- Exportar dados para análise posterior

Os gráficos são atualizados em tempo real utilizando a biblioteca `pyqtgraph`, otimizada para alta taxa de atualização sem comprometer a responsividade da interface. Um exemplo

da aplicação com teste em andamento pode ser visualizado na figura 6 abaixo:



Figura 6 – Tela da IHM desenvolvida em meio a execução de um teste. À esquerda, o gráfico de monitoramento das características térmicas do sistema e, à direita, o gráfico de monitoramento das características elétricas do mesmo.

Fonte: Elaborado pelo autor.

4 RESULTADOS E DISCUSSÃO

Neste capítulo, são apresentados os resultados obtidos com a implementação da plataforma automatizada de testes, focando na validação do controle térmico, na comparação com o sistema anterior e na demonstração da capacidade de monitoramento do sensor de envelhecimento.

4.1 Validação do Controle Térmico PID

Após realizada a modelagem do sistema térmico através do método FOPDT e, com os parâmetros obtidos a partir deste, o levantamento dos ganhos PID para o controlador, o sistema foi empiricamente refinado, a fim de gerar uma melhoria ainda mais significativa na estabilização térmica obtida.

4.1.1 Comparação: SIMC Teórico vs. Ganhos Após Fine-Tuning

Para avaliar a necessidade do refinamento empírico dos ganhos PID, realizou-se um experimento comparativo entre os ganhos calculados diretamente pelo método SIMC e os ganhos após *fine-tuning* (MORARI *et al.*, 1984; DESAI *et al.*, 2016). Ambos os controladores foram testados com setpoint de 100°C.

Tabela 4 – Comparativo de ganhos SIMC teórico vs. *fine-tuned*

Parâmetro	SIMC Teórico	Fine-Tuned	Diferença
K_p	2.78	2.78	0%
K_i	0.00231	0.00106	-54%
K_d	0	5.0	-

Fonte: Elaborado pelo autor.

A Figura 7 apresenta a análise comparativa entre os dois conjuntos de ganhos.

A análise estatística do regime permanente (últimos 30 minutos de cada teste) revelou:

Tabela 5 – Comparativo estatístico: SIMC teórico vs. *fine-tuned*

Métrica	SIMC Teórico	Fine-Tuned
Desvio Padrão (σ)	1.33 °C	0.40 °C
Overshoot Máximo	5.2 °C	0.0 °C
Tempo de Acomodação	35 min	25 min

Fonte: Elaborado pelo autor.

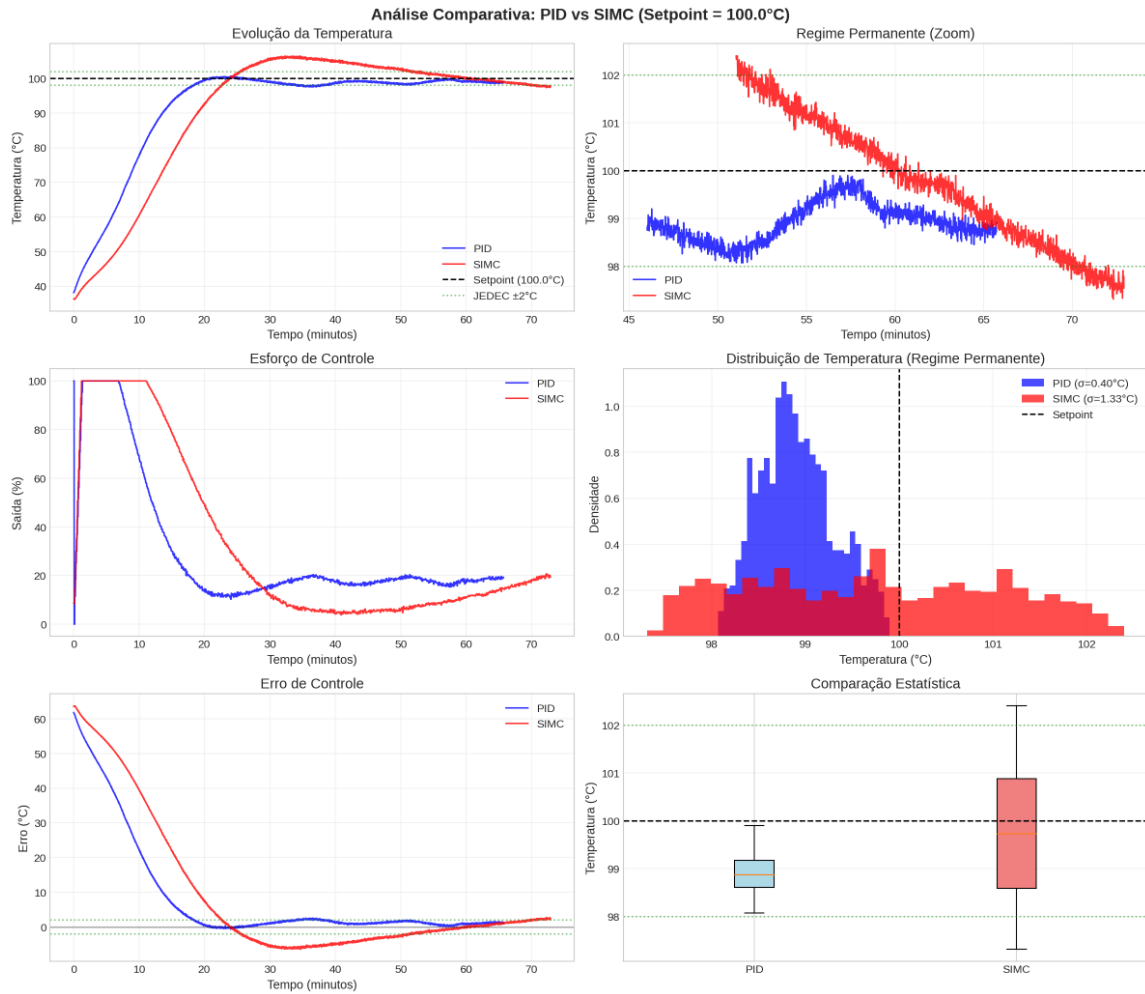


Figura 7 – Análise comparativa entre controlador SIMC puro (vermelho) e controlador após *fine-tuning* (azul) para setpoint de 100°C. (a) Evolução da temperatura; (b) Regime permanente em detalhe; (c) Esforço de controle; (d) Distribuição de temperatura em regime permanente; (e) Erro de controle; (f) Boxplot dos dois conjuntos.

A redução do ganho integral (K_i) de 0.00231 para 0.00106 diminuiu as oscilações de baixa frequência observadas com o SIMC puro. A adição da componente derivativa ($K_d = 5.0$) melhorou significativamente o amortecimento durante transientes, eliminando completamente o *overshoot* que atingia 5.2°C no SIMC teórico.

4.2 Comparação: Controle PID vs. Controle On-Off (Bang-Bang)

A fim de quantificar a melhoria proporcionada pelo controle PID em relação ao controle termostático original do forno, realizou-se um experimento comparativo direto. Ambos os sistemas foram configurados para manter 100°C por aproximadamente 1 hora. O controle On-Off (Bang-Bang) é conhecido por induzir oscilações em torno do *setpoint* devido à histerese e à inércia térmica do processo (BAI *et al.*, 2011).

4.2.1 Análise Temporal

A Figura 8 apresenta a comparação detalhada entre os dois métodos de controle.

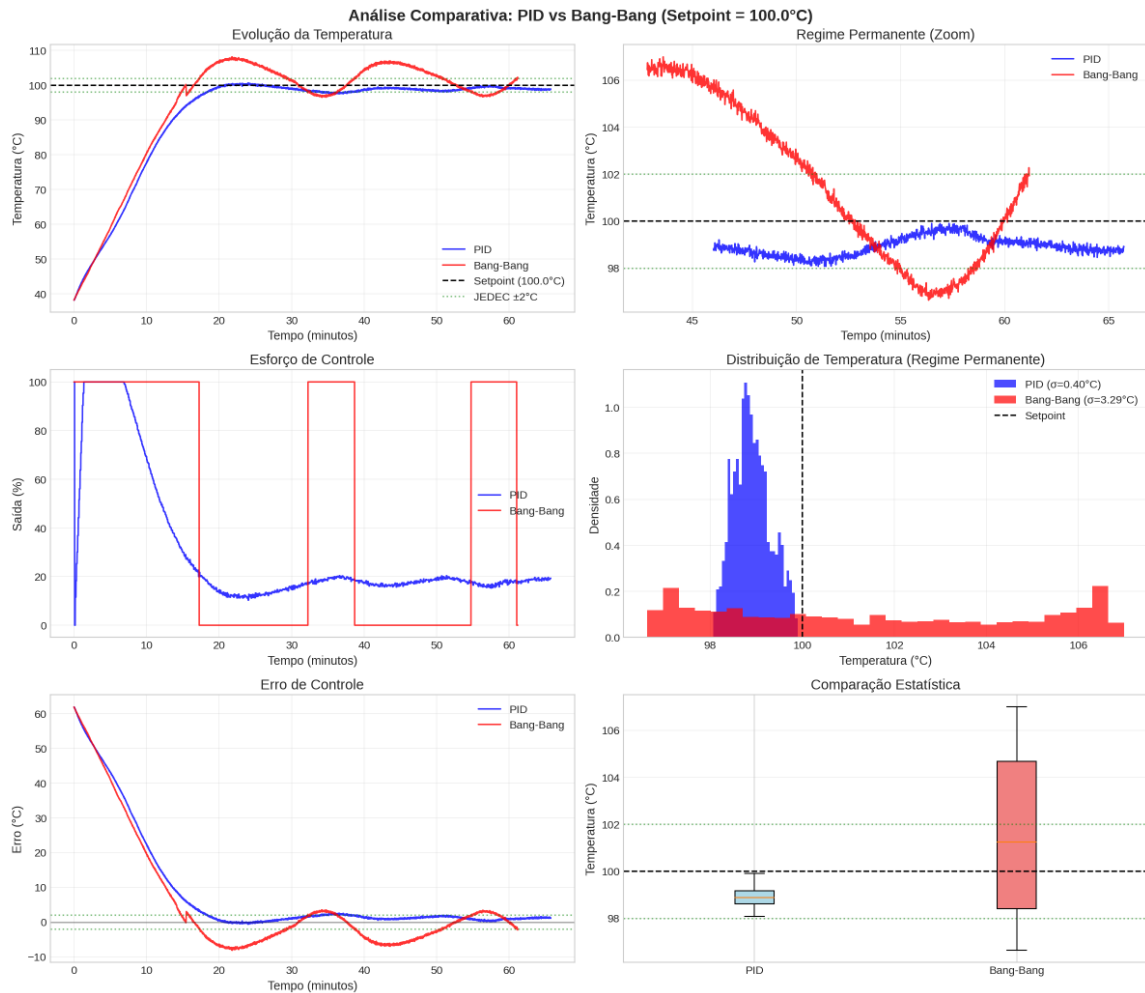


Figura 8 – Análise comparativa entre controle PID (azul) e controle Bang-Bang/On-Off (vermelho) para setpoint de 100°C. (a) Evolução da temperatura; (b) Regime permanente em detalhe; (c) Esforço de controle; (d) Distribuição de temperatura; (e) Erro de controle; (f) Boxplot dos dois conjuntos.

4.2.2 Análise Estatística da Estabilidade Térmica

Com o intuito de validar a conformidade da plataforma com os requisitos das normas JEDEC JESD22-A108 ($\pm 2^{\circ}\text{C}$) e MIL-STD-883 Método 1015 ($\pm 3^{\circ}\text{C}$) (JEDEC Solid State Technology Association, 2022; U.S. Department of Defense, 2016), foi realizada uma análise estatística dos dados de temperatura em regime permanente, representado pela Tabela 6:

Os resultados demonstram uma redução de **8.2 vezes** no desvio padrão da temperatura ($3.29^{\circ}\text{C} \rightarrow 0.40^{\circ}\text{C}$), evidenciando a superioridade do controle PID para aplicações que exigem

Tabela 6 – Comparativo estatístico de estabilidade térmica: PID vs. On-Off

Métrica	Controle On-Off	Controle PID
Desvio Padrão (σ)	3.29 °C	0.40 °C
Erro Máximo ($ e _{max}$)	7.2 °C	1.8 °C
Amplitude de Oscilação (pico-a-pico)	9.0 °C	2.1 °C
Conformidade JEDEC ($\pm 2^\circ\text{C}$)	Não	Sim
Conformidade MIL-STD ($\pm 3^\circ\text{C}$)	Não	Sim

Fonte: Elaborado pelo autor.

alta estabilidade térmica.

O controle On-Off apresenta oscilações características de “dente de serra” devido à histerese do termostato mecânico. Estas oscilações, embora aceitáveis para aplicações domésticas, introduzem ruído significativo nos dados de envelhecimento, dificultando a distinção entre degradação permanente dos transistores e variações temporárias induzidas pela temperatura.

4.3 Validação do Sistema sob Condições de Teste

O objetivo principal do sistema desenvolvido é possibilitar o teste e a validação de sensores de envelhecimento on-chip sob condições de estresse controlado. Para demonstrar esta capacidade, realizou-se um experimento de longa duração (24 horas) comparando o sistema proposto com o sistema utilizado no artigo publicado anteriormente (denominado “Sistema SBCCI”), utilizando os mesmos valores de temperatura da FPGA (115°C) e de tensão do núcleo (1.3 V) que foram utilizadas na primeira semana de envelhecimento acelerado (NOGUEIRA *et al.*, 2025). O resultado obtido é apresentado pela Figura 9.

4.3.1 Impacto na Qualidade dos Dados do Sensor

A análise da evolução do *slack* (gráfico inferior direito da Figura 9) revela um aspecto crucial: a maior estabilidade térmica do sistema PID resulta em leituras do sensor significativamente mais consistentes. A comparação estatística entre o desempenho dos dois sistemas, evidenciada pela Tabela 7, exibe de forma clara esse comportamento.

No sistema SBCCI, as oscilações térmicas de $\pm 9^\circ\text{C}$ propagam-se diretamente para as leituras do sensor, gerando uma variabilidade de $\sigma = 4.1$ incrementos. Esta variabilidade dificulta a detecção de tendências de envelhecimento de longo prazo, pois o “ruído” térmico mascara a degradação real.

No sistema PID proposto, a estabilidade de $\pm 2.6^\circ\text{C}$ reduz a variabilidade do sensor

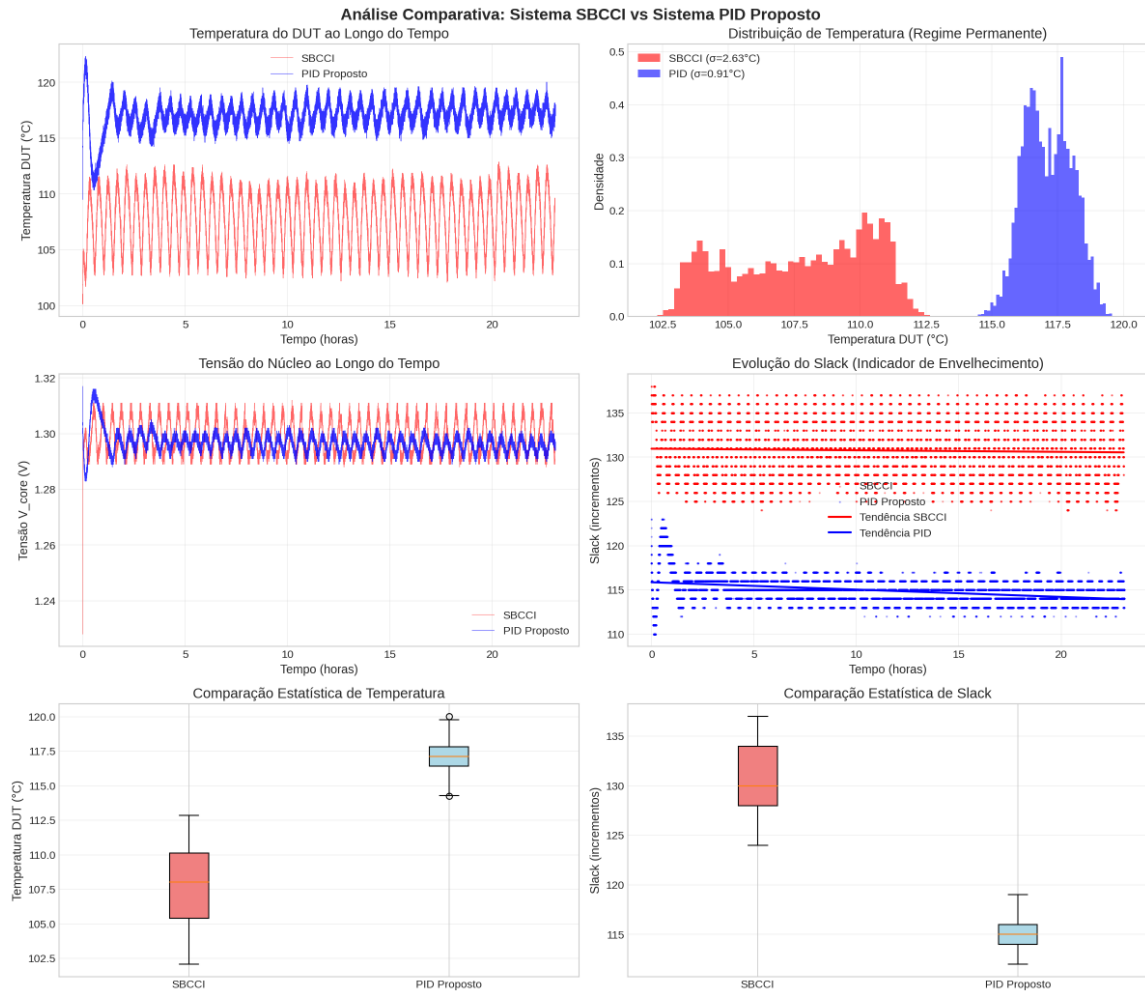


Figura 9 – Análise comparativa entre sistema SBCCI original (vermelho) e sistema PID proposto (azul) durante teste de 24 horas. (a) Temperatura do DUT ao longo do tempo; (b) Distribuição de temperatura; (c) Tensão do núcleo; (d) Evolução do *slack* (indicador de envelhecimento); (e-f) Boxplot da temperatura e do *slack* dos testes

para $\sigma = 1.8$ incrementos, permitindo identificar com maior clareza a tendência de degradação ao longo do tempo.

4.3.2 Observação sobre Temperatura Média

Nota-se que a temperatura média do DUT no sistema PID (117.2°C) é superior à do sistema SBCCI (108.2°C), mesmo com o uso de um mesmo setpoint nominal para a FPGA (115°C). Isto ocorre porque:

1. O sistema PID regula a temperatura do forno de forma consistente, tornando-o mais estável e mantendo-o mais próximo do valor desejado.
2. O sistema SBCCI oscila abaixo do setpoint, permanecendo significativo tempo abaixo do valor desejado.

Tabela 7 – Comparativo estatístico: Sistema SBCCI vs. Sistema PID Proposto (24h)

Métrica	Sistema SBCCI	Sistema PID Proposto
Temperatura do DUT		
Média	108.2 °C	117.2 °C
Desvio Padrão (σ)	2.63 °C	0.91 °C
Amplitude (pico-a-pico)	18.5 °C	5.2 °C
Leitura do Sensor (<i>Slack</i>)		
Média	130.2 incrementos	115.4 incrementos
Desvio Padrão (σ)	4.1 incrementos	1.8 incrementos

Fonte: Elaborado pelo autor.

3. A média das oscilações no sistema original resulta em uma temperatura efetiva significativamente menor do que a do sistema proposto nesse trabalho.

Esta observação destaca a necessidade do sistema anterior de uma compensação na intensidade do estresse aplicado (através do constante aumento e diminuição da tensão da FPGA), potencialmente comprometendo os cálculos de fator de aceleração baseados na equação de Arrhenius.

4.4 Integração do Sistema e Datalogging

A aplicação supervisória em Python demonstrou robustez em testes de longa duração (≈ 24 horas contínuas sem falhas). A arquitetura *multi-threading* preveniu travamentos da interface, permitindo que o usuário monitorasse os gráficos em tempo real e abortasse o teste com segurança quando necessário.

4.4.1 Estrutura dos Dados Coletados

A Tabela 8 exemplifica a estrutura dos dados gerados pelo sistema consolidado, demonstrando a sincronia temporal entre todas as variáveis monitoradas.

Tabela 8 – Exemplo de amostra de dados consolidados pelo sistema

t (s)	T_{forno}	SP	PID	V_{PSU}	I_{PSU}	T_{DUT}	Slack	V_{core}
3600	84.47	83.0	3.3%	1.450	0.89	117.2	116	1.298
3601	84.41	83.0	3.3%	1.450	0.90	117.0	116	1.299
3602	84.45	83.0	0.0%	1.450	0.91	117.4	115	1.297
3603	84.53	83.0	0.0%	1.450	0.92	117.1	116	1.298

Fonte: Elaborado pelo autor. Unidades: T em °C, V em V, I em A.

A sincronia temporal entre a tensão da fonte (V_{PSU}), a temperatura do forno (T_{forno}),

a temperatura do *die* (T_{DUT}) e a leitura do sensor (*Slack*) elimina a incerteza de medição que existia no método manual, permitindo análises de correlação precisas.

4.4.2 Análise de Segurança

Durante os testes de validação, foram simuladas diversas condições de falha para verificar o correto funcionamento dos mecanismos de segurança:

1. **Desconexão USB:** O firmware do Arduino, ao detectar a ausência de comandos de *heartbeat* do software Python por mais de 5 segundos, atuou corretamente desligando o pino do SSR (estado *Fail-Safe*).
2. **Limite de corrente:** Ao simular um aumento anormal de corrente (reduzindo a resistência de carga), a fonte IT6502D desligou automaticamente via proteção OCP, demonstrando a redundância do sistema.
3. **Temperatura máxima:** Ao desabilitar temporariamente o controle PID e permitir aquecimento livre, o firmware acionou o limite absoluto ($T > 130^{\circ}\text{C}$) e desligou o aquecimento antes de atingir temperaturas perigosas.

Estes testes comprovam que o sistema possui mecanismos de proteção adequados para operação autônoma (*unattended*), reduzindo o risco de incêndio ou dano catastrófico ao equipamento em testes de longa duração.

5 CONCLUSÕES E TRABALHOS FUTUROS

Este capítulo sintetiza as principais contribuições do trabalho, avalia o cumprimento dos objetivos propostos e apresenta sugestões para a continuidade da pesquisa.

5.1 Síntese do Trabalho

O presente trabalho teve como motivação as limitações identificadas durante a validação experimental do sensor de envelhecimento com auto-calibração, publicado por Nogueira *et al.* (NOGUEIRA *et al.*, 2025). Naquele contexto, a utilização de um forno de convecção convencional com controle termostático e operação manual da fonte de alimentação resultou em instabilidades térmicas que comprometiam a precisão dos dados de envelhecimento e a conformidade com as normas industriais (U.S. Department of Defense, 2016; JEDEC Solid State Technology Association, 2022).

Para superar essas limitações, foi desenvolvida uma plataforma automatizada de testes de envelhecimento acelerado (*burn-in*) que integra:

- Um sistema de controle térmico em malha fechada baseado em algoritmo PID digital
- Uma aplicação supervisória com arquitetura *multi-threading* para orquestração simultânea de múltiplos instrumentos
- Mecanismos redundantes de segurança para operação autônoma de longa duração

Os resultados experimentais demonstraram que a plataforma desenvolvida atinge estabilidade térmica de $\pm 0.8^{\circ}\text{C}$ (desvio padrão de 0.40°C em regime permanente), superando significativamente os requisitos das normas MIL-STD-883 Método 1015 ($\pm 3^{\circ}\text{C}$) e JEDEC JESD22-A108 ($\pm 2^{\circ}\text{C}$) (U.S. Department of Defense, 2016; JEDEC Solid State Technology Association, 2022). Esta precisão representa uma melhoria de **8.2 vezes** em relação ao sistema anterior, validando a viabilidade de construir equipamentos de teste de nível industrial utilizando componentes de baixo custo.

5.2 Avaliação dos Objetivos

5.2.1 Objetivo Geral

O objetivo geral de *projetar, implementar e validar uma plataforma automatizada de testes de confiabilidade para FPGAs, capaz de controlar temperatura e tensão com alta precisão*

foi alcançado. A plataforma desenvolvida demonstrou capacidade de:

- Controlar a temperatura do forno com precisão de $\pm 0.8^{\circ}\text{C}$
- Regular a tensão de alimentação da FPGA via comandos SCPI com resolução de 1 mV
- Coletar dados sincronizados de todas as variáveis relevantes (temperatura, tensão, corrente, *slack* do sensor)
- Operar de forma autônoma por períodos superiores a 24 horas sem falhas

5.2.2 *Objetivos Específicos*

A Tabela 9 apresenta a avaliação do cumprimento de cada objetivo específico proposto na introdução deste trabalho.

Tabela 9 – Avaliação do cumprimento dos objetivos específicos

Objetivo Específico	Status	Seção
1. Desenvolver sistema de controle térmico em malha fechada com Arduino Nano 33 IoT e algoritmo PID sintonizado pelo método SIMC	Alcançado	3.3
2. Projetar e montar interface de potência com SSR e sensores NTC ratiométricos	Alcançado	3.2
3. Desenvolver aplicação supervisória em Python com arquitetura <i>multi-threading</i>	Alcançado	3.6
4. Implementar rotinas de segurança e perfis de rampa de temperatura	Alcançado	3.4
5. Validar o sistema comparando sua estabilidade com os requisitos das normas JEDEC e MIL-STD-883	Alcançado	4.2

Fonte: Elaborado pelo autor.

5.3 Contribuições do Trabalho

As principais contribuições técnicas e científicas deste trabalho são:

5.3.1 *Estabilidade Térmica de Precisão Industrial*

A implementação do controle PID com sintonia SIMC, combinada com a leitura ratiométrica de 12 bits do sensor NTC, permitiu atingir uma estabilidade térmica ($\sigma = 0.40^{\circ}\text{C}$) que alcança, de forma satisfatória, os requisitos normativos. Esta contribuição valida a hipótese de que hardware de baixo custo (Arduino, termistor NTC, relé SSR), quando adequadamente projetado e calibrado, pode entregar desempenho comparável a equipamentos industriais de

custo significativamente superior.

5.3.2 *Atenuação do Ruído Térmico nos Dados de Envelhecimento*

A comparação experimental entre o sistema proposto e o sistema utilizado no artigo anterior (Sistema SBCCI) demonstrou que a maior estabilidade térmica resulta em leituras do sensor de envelhecimento significativamente mais consistentes. A redução do desvio padrão da temperatura do DUT de 2.63°C para 0.91°C refletiu-se diretamente na redução da variabilidade das leituras do sensor de 4.1 para 1.8 incrementos.

Esta melhoria é fundamental para a detecção de tendências de envelhecimento de longo prazo, pois permite separar a degradação permanente dos transistores (causada por BTI e HCI) das variações temporárias induzidas pela oscilação térmica.

5.3.3 *Automação e Repetibilidade*

A substituição da operação manual por um software supervisor centralizado trouxe benefícios significativos:

- **Repetibilidade:** Perfis de teste idênticos podem ser executados múltiplas vezes, eliminando a variabilidade introduzida pelo operador humano
- **Rastreabilidade:** Todos os parâmetros do teste (setpoints, ganhos PID, limites de segurança) são registrados automaticamente no cabeçalho dos arquivos de *log*
- **Sincronia:** A coleta simultânea de dados de múltiplas fontes (temperatura, tensão, sensor) com *timestamps* unificados elimina incertezas de correlação temporal

5.3.4 *Segurança para Operação Autônoma*

A implementação de camadas redundantes de segurança (WDT de hardware, *heart-beat* de software, limites absolutos de temperatura, proteção de sobrecorrente) permite a execução de testes de longa duração sem supervisão humana constante. Esta característica é essencial para testes de *burn-in* que podem durar centenas ou milhares de horas conforme especificado pelas normas JEDEC (JEDEC Solid State Technology Association, 2022).

5.4 Melhorias no Sistema de Controle

1. **Controle em Cascata:** Implementar uma arquitetura de controle em cascata onde a malha externa controla a temperatura do DUT (medida pelo XADC) e a malha interna controla a temperatura do forno. Esta abordagem compensaria automaticamente o offset variável entre as duas temperaturas.
2. **Controle Adaptativo:** Investigar técnicas de controle adaptativo (ex: *Model Reference Adaptive Control*) para ajustar automaticamente os ganhos do PID conforme as condições de operação mudam (potência dissipada pela FPGA, temperatura ambiente).
3. **Aumento da Potência de Aquecimento:** Substituir a resistência de aquecimento original por uma de maior potência ou adicionar isolamento térmico para permitir operação em temperaturas mais elevadas.

5.5 Expansão de Funcionalidades

1. **Controle de Umidade:** Integrar sensores e atuadores de umidade para permitir a realização de testes HAST (*Highly Accelerated Stress Test*), conforme a norma JEDEC JESD22-A110, possibilitando o estudo de mecanismos de falha relacionados à corrosão e absorção de umidade pelo encapsulamento.
2. **Escalabilidade Multi-DUT:** Expandir o hardware e software para permitir o teste simultâneo de múltiplas FPGAs em paralelo. Isto aumentaria a significância estatística dos dados coletados e reduziria o tempo total necessário para caracterização de lotes de dispositivos.
3. **Ciclagem Térmica:** Implementar perfis de ciclagem térmica automática para testes de fadiga termomecânica, complementando os testes de vida estática (*steady-state life*) atualmente suportados.

5.6 Análise Avançada de Dados

1. **Predição de Vida Útil:** Utilizar os dados de envelhecimento coletados para treinar modelos de *Machine Learning* capazes de prever a Vida Útil Restante (*Remaining Useful Life - Remaining Useful Life (RUL)*) dos dispositivos. Esta capacidade habilitaria estratégias de manutenção preditiva em sistemas críticos.
2. **Caracterização de Mecanismos de Falha:** Correlacionar os dados do sensor de envelhecimento com medições elétricas paramétricas (corrente de fuga, tensão de limiar)

para identificar e separar os diferentes mecanismos de degradação (NBTI, *Positive Bias Temperature Instability* (PBTI), HCI).

5.7 Considerações Finais

Este trabalho demonstrou que é viável desenvolver uma plataforma de testes de confiabilidade de circuitos integrados que combina precisão industrial com custo acessível para laboratórios acadêmicos. A integração entre técnicas clássicas de engenharia de controle (PID, modelagem FOPDT, sintonia SIMC) e ferramentas modernas de desenvolvimento de software (Python, Qt, arquitetura *multi-threading*) resultou em um sistema robusto e flexível.

A plataforma desenvolvida não apenas supera as limitações do sistema anterior, mas estabelece uma base sólida para futuras pesquisas em confiabilidade de semicondutores e *Silicon Lifecycle Management*. Os arquivos de código-fonte (firmware e software supervisorio) disponibilizados nos apêndices deste trabalho permitem a reprodução e adaptação do sistema para outras aplicações de teste e caracterização.

REFERÊNCIAS

- AHMED, F.; SHINTANI, M.; INOUE, M. Accurate recycled fpga detection using an exhaustive-fingerprinting technique. **IEEE Trans. CAD of Integrated Circuits and Systems**, v. 40, n. 8, p. 1626–1639, 2021.
- AMD Xilinx. **7 Series FPGAs Clocking Resources User Guide**. [S.l.], 2023.
- AMD Xilinx. **Artix-7 FPGA Data Sheet: DC and AC Switching Characteristics**. [S.l.], 2023.
- AMOURI, A.; EBRAHIMI, M.; TAHOORI, M. High-level aging estimation for fpga-mapped designs. In: **22nd Int. Conf. Field Programmable Logic and Applications (FPL)**. [S.l.: s.n.], 2012. p. 284–291.
- AMOURI, A. *et al.* Aging effects in fpgas: an experimental analysis. In: **24th Int. Conf. Field Programmable Logic and Applications (FPL)**. [S.l.: s.n.], 2014. p. 1–4.
- BAI, B. *et al.* Thermal management of integrated circuits in burn-in environment. In: **IEEE International Conference**. [S.l.: s.n.], 2011. p. 1092–1095.
- BENFICA, J. *et al.* Analysis of sram-based fpga seu sensitivity to combined emi and tid-imprinted effects. **IEEE Trans. Nuclear Science**, v. 63, n. 2, p. 1294–1300, 2016.
- DESAI, R.; BHOR, M.; ADMUTHE, S. Performance assessment of model-based simc for fopdt system. In: **3rd International Conference on Electrical, Electronics, Engineering Trends, Communication, Optimization and Sciences (EEECOS 2016)**. [S.l.: s.n.], 2016. p. 1–4.
- GHADERI, Z. *et al.* Sensible: A highly scalable sensor design for path-based age monitoring in fpgas. **IEEE Trans. Computers**, v. 66, n. 5, p. 919–926, 2017.
- JEDEC Solid State Technology Association. **Temperature, Bias, and Operating Life**. 2022.
- LANZIERI, L. *et al.* A review of techniques for ageing detection and monitoring on embedded systems. **ACM Computing Surveys**, v. 57, n. 1, 2024.
- MORARI, M.; SKOGESTAD, S.; RIVERA, D. F. Implications of internal model control for pid controllers. In: **1984 American Control Conference**. [S.l.: s.n.], 1984. p. 661–666.
- NOGUEIRA, L. *et al.* Auto-tuning aging sensor validated under burn-in, temperature, and voltage variations. p. 1–5, 2025.
- U.S. Department of Defense. **MIL-STD-883, Method 1005.9: Steady-state life**. 2010.
- U.S. Department of Defense. **MIL-STD-883, Method 1015: Burn-In Test**. 2016.
- VALDES-PENA, M. *et al.* Design and validation of configurable online aging sensors in nanometer-scale fpgas. **IEEE Trans. Nanotechnology**, v. 12, n. 4, p. 508–517, 2013.
- VARGAS, F.; BALAKRISHNAN, A. On-chip aging sensor core for silicon lifecycle management. In: **IEEE 26th Latin American Test Symposium (LATS)**. [S.l.: s.n.], 2025. p. 1–6.
- VARGAS, F. *et al.* On-chip sensor to monitor aging evolution in finfet-based memories. In: **IEEE 30th Int. Symp. On-Line Testing and Robust System Design (IOLTS)**. [S.l.: s.n.], 2024. p. 1–6.

VASSIGHI, A.; SEMENOV, O.; SACHDEV, M.; KESHAVARZI, A. Thermal management of high performance microprocessors in burn-in environment. In: **IEEE Proceedings**. [S.l.: s.n.], 2004.

XU, J.; REN, H.; YAN, B.; LI, Z. Design of temperature control system based on stm32 and tuned pid algorithm. In: **IEEE 4th Int. Conf. Information Technology, Big Data and AI (ICIBA)**. [S.l.: s.n.], 2024. p. 112–116.

Apêndices

APÊNDICE A – FIRMWARE DO CONTROLADOR TÉRMICO (ARDUINO NANO)

Este apêndice apresenta trechos essenciais do firmware embarcado no microcontrolador Arduino Nano 33 IoT, responsável pelo controle térmico do forno de *burn-in*. O código-fonte completo está disponível no repositório do projeto em: <https://drive.google.com/drive/folders/1BJ9GEoLW_C_ueiGxISTA7ixx8njzxTym?usp=sharing>.

Código-fonte 1 – Parâmetros do sistema e inicialização do PID (ArduPID)

```

1  #include <ArduPID.h>
2
3  // ADC e NTC
4  const int    ADC_MAX_BITS    = 12;
5  const int    ADC_SAMPLES     = 500;
6
7  // Controle
8  const long   CONTROL_PERIOD_MS = 5000;
9  const float  RAMP_RATE_PER_SEC = 1.0f;
10
11 // Ganhos PID (calculados via SIMC e refinados
    empiricamente)
12 const double KP = 2.78;
13 const double KI = 0.00106;
14 const double KD = 5.0;
15
16 ArduPID myPID;
17
18 void setup() {
19     {...}
20
21     myPID.begin(&currentCelsius, &pidOutput, &rampedSetpoint,
        KP, KI, KD);
22     myPID.setOutputLimits(0, 100);
23     myPID.setWindUpLimits(0, 100);

```

```

24 myPID.setSampleTime(CONTROL_PERIOD_MS);
25
26 {...}
27 }

```

Código-fonte 2 – Leitura do NTC com média de amostras e conversão por equação Beta

```

1 float getOvenTemperature() {
2     float adc_avg = adc_read_averaged(NTC_PIN, ADC_SAMPLES);
3     float r_ntc    = ntc_resistance_from_adc(adc_avg);
4     float celsius  = ntc_temp_from_resistance(r_ntc);
5     return celsius;
6 }
7
8 float ntc_temp_from_resistance(float R_ntc) {
9     const float TO_K = NTC_T_NOMINAL_C + 273.15f;
10    float invT = 1.0f / TO_K + (1.0f / NTC_B_VALUE) * log(
11        R_ntc / NTC_R_NOMINAL);
12    float T_K = 1.0f / invT;
13    return T_K - 273.15f;
14 }

```

Código-fonte 3 – Aplicação de rampa no setpoint (atualização a cada 1 s)

```

1 unsigned long rampTimer = 0;
2
3 void updateSetpointRamp() {
4     unsigned long currentMillis = millis();
5
6     if (currentMillis - rampTimer >= 1000) {
7         rampTimer = currentMillis;
8     }
9 }

```

```

9      float maxChange = RAMP_RATE_PER_SEC * 1.0;
10
11     if (targetSetpoint > rampedSetpoint) {
12         rampedSetpoint += maxChange;
13         if (rampedSetpoint > targetSetpoint) rampedSetpoint =
            targetSetpoint;
14     } else if (targetSetpoint < rampedSetpoint) {
15         rampedSetpoint -= maxChange;
16         if (rampedSetpoint < targetSetpoint) rampedSetpoint =
            targetSetpoint;
17     }
18 }
19 }

```

Código-fonte 4 – Protocolo serial com o supervisor (comandos e telemetria)

```

1 void handleSerialCommands() {
2     if (Serial.available() > 0) {
3         {...}
4         if (cmd.startsWith("SET_SP,")) {
5             targetSetpoint = cmd.substring(7).toFloat();
6             Serial.println("OK,SP_SET");
7         }
8         else if (cmd.equals("START_TEST")) {
9             currentCelsius = getOvenTemperature();
10            rampedSetpoint = currentCelsius;
11
12            myPID.reset();
13            myPID.start();
14            {...}
15            Serial.println("OK,TEST_STARTED");
16        }

```

```

17     else if (cmd.equals("STOP_TEST")) {
18         {...}
19         testRunning = false;
20         digitalWrite(SSR_PIN, LOW);
21         {...}
22         Serial.println("OK, TEST_STOPPED");
23     }
24     else if (cmd.equals("GET_DATA")) {
25         {...}
26     }
27     {...}
28 }
29 }

```

Código-fonte 5 – Loop de acionamento do SSR por *time-proportioning* (janela de 5 s)

```

1
2 void loop() {
3     handleSerialCommands();
4
5     if (!testRunning) {
6         digitalWrite(SSR_PIN, LOW);
7         return;
8     }
9
10    currentCelsius = getOvenTemperature();
11    updateSetpointRamp();
12    myPID.compute();
13    runSsrControl(pidOutput);
14 }

```

APÊNDICE B – DRIVERS E ORQUESTRAÇÃO DO ENSAIO (PYTHON WORKERS)

Este apêndice apresenta um resumo dos componentes em Python responsáveis pela comunicação com os instrumentos (Arduino, fonte programável e DUT/FPGA) e pela orquestração do ensaio (*sequencer*). O objetivo é evidenciar a **arquitetura, interfaces e fluxo de execução** sem reproduzir integralmente o código, uma vez que a versão completa está disponível no repositório do projeto em: <<https://drive.google.com/drive/folders/1wHy7OzRENjjd7lcIwGLLhHmJSE1NRfQZ?usp=sharing>>.

Interfaces (assinaturas) e fluxo principal

Código-fonte 6 – Esqueleto dos *workers*: instrumentos e sinais principais

```

1  # workers.py
2
3  class ArduinoWorker(QObject):
4      log_message: Signal(str)
5      data_ready: Signal(float, float, float) # (T_forno,
6          SP_rampa, saida_%)
7
8      def start(self) -> None: ...
9      def stop(self) -> None: ...
10     def poll_data(self) -> None: ...
11     def send_command(self, cmd: str) -> str: ...
12
13     # API sem ntica utilizada pelo sequenciador
14     def set_target_setpoint(self, sp_celsius: float) -> str
15         : ...
16     def start_test_oven(self) -> str: ...
17     def stop_test_oven(self) -> str: ...
18     def get_latest_data(self) -> tuple[float, float, float
19         ]: ...

```



```

18 class PSUWorker(QObject):
19     log_message: Signal(str)
20     data_ready: Signal(float, float) # (V, I)
21
22     def start(self) -> None: ...
23     def stop(self) -> None: ...
24     def poll_data(self) -> None: ...
25
26     # Controle via SCPI/PyVISA
27     def set_voltage(self, v: float) -> None: ...
28     def turn_on(self) -> None: ...
29     def turn_off(self) -> None: ...
30     def get_latest_data(self) -> tuple[float, float]: ...
31
32 class DUTWorker(QObject):
33     log_message: Signal(str)
34     data_ready: Signal(float, int, float) # (T_DUT, slack
35         , V_DUT)
36
37     def start(self) -> None: ...
38     def stop(self) -> None: ...
39     def poll_data(self) -> None: ...
40     def get_latest_data(self) -> tuple[float, int, float]:
41         ...

```

Código-fonte 7 – Esqueleto do *sequencer*: coordenação, log e segurança

```

1 # test_sequencer.py
2
3 class TestSequencer(QObject):
4     log_message: Signal(str)
5     plot_data_update: Signal(dict)

```

```

6     test_finished: Signal()
7
8     def __init__(self, arduino: ArduinoWorker, psu:
9         PSUWorker, dut: DUTWorker): ...
10
11     def start_test(self, settings: dict) -> None:
12         # 1) inicializa logger (CSV) e registra
13             configuracoes
14         # 2) configura setpoints (forno e fonte) e liga PSU
15         # 3) inicia controle no Arduino e habilita loop de
16             aquisicao/log
17         ...
18
19     def stop_test(self) -> None:
20         # desabilita Arduino + PSU, fecha logger, emite
21             sinal de termino
22         ...
23
24     def log_data_tick(self) -> None:
25         # coleta dados mais recentes de todos os workers
26         # monta dicionario e registra em CSV
27         # atualiza graficos via plot_data_update
28         # aplica intertravamentos
29         ...
30
31     def _check_safety_limits(self, t_dut: float, i_psu:
32         float, t_oven: float) -> None:
33         # if (t_dut > MAX_DUT_TEMP) or (i_psu > MAX_CURRENT
34             ) or (t_oven > MAX_OVEN_TEMP):
35         #     stop_test()
36         ...

```

Código-fonte 8 – Registro de dados: contrato mínimo do logger (CSV)

```
1 # logger.py
2
3 class DataLogger:
4     def __init__(self, folder: str, test_name: str): ...
5     def write_data_row(self, row: dict) -> None: ...
6     def close(self) -> None: ...
```

APÊNDICE C – INTERFACE GRÁFICA DA IHM (PYTHON PYSIDE6)

Este apêndice apresenta um resumo do monitoramento em Python (PySide6), destacando a organização em *threads*, a comunicação por sinais/*slots* e o fluxo de operação do ensaio. O código-fonte completo está disponível em: <<https://drive.google.com/drive/folders/1wHy7OzRENj7lcIwGLLhHmJSE1NRfQZ?usp=sharing>>.

Arquitetura da aplicação

- `App.py`: ponto de entrada da aplicação (criação do `QApplication`).
- `main_window.py`: janela principal, inicialização de *threads* e elementos de UI.
- `workers.py`: *workers* por instrumento, executando em `QThread`.
- `test_sequencer.py`: coordenação do ensaio, log e segurança.

Código-fonte 9 – Ponto de entrada

```

1 # App.py
2
3 def main() -> None:
4     # app = QApplication(sys.argv)
5     # window = MainWindow()
6     # window.show()
7     # sys.exit(app.exec())
8     ...

```

Código-fonte 10 – Inicialização de *threads* e *workers*

```

1 # main_window.py
2
3 class MainWindow(QMainWindow):
4     def _start_worker(self, name: str,
5                       worker_cls_or_instance):
6         # cria QThread
7         # move worker para a thread
8         # conecta sinais de log/dados

```

```

8         # inicia thread
9         ...
10
11     def _start_device_workers(self) -> None:
12         # arduino_worker = _start_worker("arduino",
13             ArduinoWorker)
14         # psu_worker      = _start_worker("psu",      PSUWorker
15             )
16         # dut_worker      = _start_worker("dut",      DUTWorker
17             )
18         ...

```

Código-fonte 11 – Orquestração do ensaio e atualização de UI

```

1 # main_window.py + test_sequencer.py
2
3 class MainWindow(QMainWindow):
4     def _start_test_sequencer(self) -> None:
5         # sequencer = TestSequencer(arduino_worker,
6             psu_worker, dut_worker)
7         # conecta start/stop do botao principal aos metodos
8             do sequencer
9         # conecta plot_data_update a widgets de grafico
10        # conecta test_finished a rotinas de finalizacao
11        ...
12
13     def on_toggle_test(self, checked: bool) -> None:
14         # if checked:
15         #     settings = {nome, setpoint_forno,
16             tensao_fonte, ...}
17         #     start_test_signal.emit(settings)
18         # else:

```

```
16         #         stop_test_signal.emit()  
17         ...
```

Código-fonte 12 – Contrato mínimo de atualização de gráficos e log na UI

```
1 # widgets.py / main_window.py  
2  
3 def update_plot_data(data: dict) -> None:  
4     # recebe dicionario padronizado:  
5     # {time_sec, oven_temp, oven_setpoint, oven_output,  
6         psu_voltage, psu_current, dut_temp, dut_slack,  
7         dut_volt}  
8     # atualiza curvas e indicadores  
9     ...  
10  
11 def log_message(msg: str) -> None:  
12     # append com carimbo de tempo  
13     ...
```